

---

# **Synca XFS 一体化臂手总成 SDK 用户手册 V1.2**

杭州曦诺未来科技有限公司

2026 年 9 月

# 目录

|                         |           |
|-------------------------|-----------|
| <b>1 手册概述</b>           | <b>1</b>  |
| 1.1 关于本手册               | 1         |
| 1.2 手册对象                | 1         |
| 1.3 版本记录                | 1         |
| <b>2 免责声明</b>           | <b>2</b>  |
| <b>3 概述</b>             | <b>3</b>  |
| 3.1 兼容性                 | 3         |
| 3.1.1 控制器版本和机器人型号       | 3         |
| 3.1.2 运行环境与语言           | 3         |
| 3.2 非实时控制               | 3         |
| 3.3 实时控制                | 4         |
| <b>4 使用指南</b>           | <b>5</b>  |
| 4.1 设置                  | 5         |
| 4.1.1 硬件设置              | 5         |
| 4.1.2 网络配置              | 5         |
| 4.1.3 机器人功能设置           | 5         |
| 4.2 编译                  | 6         |
| 4.2.1 xynova SDK 工程包说明  | 6         |
| 4.2.2 C++ 编译            | 6         |
| 4.2.2.1 Linux 平台编译      | 6         |
| 4.3 本地配置文件              | 6         |
| 4.3.1 运行日志              | 6         |
| 4.3.2 实时模式等待超时时间设置      | 7         |
| 4.4 语言                  | 7         |
| 4.5 Linux 实时环境配置（可选）    | 7         |
| <b>5 接口说明</b>           | <b>10</b> |
| 5.1 对象模型与通用约定           | 10        |
| 5.1.1 对象模型              | 10        |
| 5.2 Robot 整机接口          | 11        |
| 5.2.1 构造与生命周期           | 11        |
| 5.2.2 连接管理              | 11        |
| 5.2.3 信息聚合              | 11        |
| 5.2.4 整机安全协调            | 12        |
| 5.2.5 日志管理              | 12        |
| 5.2.6 固件管理              | 12        |
| 5.2.7 臂手协同              | 12        |
| 5.2.8 运动学解算             | 13        |
| 5.2.9 子系统访问器            | 13        |
| 5.3 Arm 机械臂接口           | 13        |
| 5.3.1 构造与生命周期           | 13        |
| 5.3.2 连接管理              | 13        |
| 5.3.3 状态查询              | 14        |
| 5.3.4 固件 / 备份 / 时钟      | 15        |
| 5.3.5 模式与使能             | 15        |
| 5.3.6 运动控制              | 15        |
| 5.3.7 运动学 / 标定 / 软限位    | 16        |
| 5.3.8 基坐标系 / 控制器日志 / 系统 | 16        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| 5.3.9 状态 / 急停恢复                                     | 17        |
| 5.3.10 实时轨迹跟随                                       | 17        |
| 5.3.11 ServoJ: 实时轴空间伺服位置控制                          | 17        |
| 5.3.12 力传感器标定 / 力控参数 / 默认速度                         | 17        |
| 5.3.13 RT 网络容忍度 / 状态反馈                              | 18        |
| 5.3.14 ServoJ 滤波 / RT 回调循环                          | 18        |
| 5.3.15 轴空间阻抗控制                                      | 18        |
| 5.3.16 轴空间力矩控制                                      | 19        |
| 5.3.17 轴空间 Jog                                      | 19        |
| 5.4 Hand 灵巧手接口                                      | 19        |
| 5.4.1 构造与生命周期                                       | 19        |
| 5.4.2 连接管理                                          | 19        |
| 5.4.3 设备信息                                          | 20        |
| 5.4.4 状态                                            | 20        |
| 5.4.5 使能 / 运动控制                                     | 20        |
| 5.4.6 传感器使能                                         | 20        |
| 5.4.7 传感器读取                                         | 21        |
| 5.4.8 故障恢复                                          | 21        |
| 5.5 错误码                                             | 21        |
| 5.5.1 形态                                            | 21        |
| 5.5.2 错误码全表                                         | 21        |
| 5.5.3 错误来源 ErrorSource                              | 24        |
| <b>6 注意事项与问题排查</b>                                  | <b>25</b> |
| 6.1 实时指令                                            | 25        |
| 6.1.1 轴空间运动                                         | 25        |
| 6.1.2 力矩直接控制                                        | 26        |
| 6.1.3 运动限制条件                                        | 26        |
| 6.2 问题排查                                            | 26        |
| 6.2.1 网络连接问题                                        | 26        |
| 6.2.2 实时模式直接力矩控制下坠问题                                | 26        |
| <b>7 使用示例</b>                                       | <b>27</b> |
| 7.1 只读巡检                                            | 27        |
| 7.2 机械臂运动 (运动前确认安全)                                 | 28        |
| 7.3 灵巧手控制                                           | 28        |
| 7.4 臂手协同                                            | 28        |
| 7.5 更多可运行示例                                         | 28        |
| <b>8 反馈与勘误</b>                                      | <b>29</b> |
| <b>9 附录 A - C++ API</b>                             | <b>30</b> |
| 9.1 配置 (RobotConfig)                                | 30        |
| 9.2 枚举参考                                            | 31        |
| 9.2.1 ConnectionStatus (C: ArmhandConnectionStatus) | 31        |
| 9.2.2 MotionState (C: ArmhandMotionState)           | 31        |
| 9.2.3 RtActiveMode (C: ArmhandRtActiveMode)         | 32        |
| 9.2.4 RtStateField (C: ArmhandRtStateField)         | 32        |
| 9.2.5 OperatingMode (C: ArmhandOperatingMode)       | 32        |
| 9.2.6 DragSpaceMode (C: ArmhandDragSpaceMode)       | 33        |
| 9.2.7 LogCleanupMode (C: ArmhandLogCleanupMode)     | 33        |
| 9.2.8 SyncStrategy (C: ArmhandSyncStrategy)         | 33        |
| 9.2.9 HandControlMode (C: ArmhandHandControlMode)   | 33        |
| 9.2.10 LogLevel (C: ArmhandLogLevel)                | 34        |
| 9.2.11 OperationState (C: ArmhandOperationState)    | 34        |
| 9.2.12 PowerState (C: ArmhandPowerState)            | 34        |

|                                                                  |    |
|------------------------------------------------------------------|----|
| 9.2.13 MotionControlMode (C: ArmhandMotionControlMode)           | 35 |
| 9.2.14 RecoverOption (C: ArmhandRecoverOption)                   | 35 |
| 9.2.16 BackupItem (C: ArmhandBackupItem)                         | 35 |
| 9.2.17 StopLevel (C: ArmhandStopLevel)                           | 36 |
| 9.2.18 FirmwareUpdateState (C: ArmhandFirmwareUpdateState)       | 36 |
| 9.3 数据结构                                                         | 37 |
| 9.3.1 Pose (C: ArmhandPose)                                      | 37 |
| 9.3.2 JointState (C: ArmhandJointState)                          | 37 |
| 9.3.3 JointPosition (C: ArmhandJointPosition)                    | 37 |
| 9.3.4 FingerState (C: ArmhandFingerState)                        | 37 |
| 9.3.5 WorldFrame (C: ArmhandWorldFrame)                          | 38 |
| 9.3.6 CollisionState (C: ArmhandCollisionState)                  | 38 |
| 9.3.7 ErrorInfo (C: ArmhandErrorInfo)                            | 38 |
| 9.3.8 VersionInfo (C: ArmhandVersionInfo)                        | 38 |
| 9.3.9 HandSchedulingToggles (C: ArmhandHandSchedulingToggles)    | 39 |
| 9.3.10 DevicePartInfo (C: ArmhandDevicePartInfo)                 | 39 |
| 9.3.11 DevicePartError (C: ArmhandDevicePartError)               | 39 |
| 9.3.12 DeviceInfo (C: ArmhandDeviceInfo)                         | 40 |
| 9.3.13 ErrorState (C: ArmhandErrorState)                         | 40 |
| 9.3.14 DeviceConnectionStatus (C: ArmhandDeviceConnectionStatus) | 40 |
| 9.3.15 TactileWrench (C: ArmhandTactileWrench)                   | 40 |
| 9.3.16 TactileTaxel (C: ArmhandTactileTaxel)                     | 40 |
| 9.3.17 RangePoint (C: ArmhandRangePoint)                         | 41 |
| 9.3.18 ArmTarget (C: ArmhandArmTarget)                           | 41 |
| 9.3.19 HandTarget (C: ArmhandHandTarget)                         | 41 |
| 9.3.20 RobotArmState (C: ArmhandRobotArmState)                   | 41 |
| 9.3.21 RobotHandState (C: ArmhandRobotHandState)                 | 42 |
| 9.3.22 RobotSyncState (C: ArmhandRobotSyncState)                 | 42 |
| 9.3.23 RobotState (C: ArmhandRobotState)                         | 42 |
| 9.3.25 SoftLimitConfig (C: ArmhandSoftLimitConfig)               | 43 |
| 9.3.26 SafetyStatus (C: ArmhandSafetyStatus)                     | 43 |
| 9.3.27 HandConfig (C: ArmhandHandConfig)                         | 43 |
| 9.3.28 ErrorConfig (C: ArmhandErrorConfig)                       | 44 |
| 9.3.29 RobotConfig (C: ArmhandRobotConfig)                       | 44 |
| 9.3.30 FirmwareUpdateConfig                                      | 44 |
| 9.3.31 FirmwareUpdateProgress (C: ArmhandFirmwareUpdateProgress) | 44 |
| 9.3.32 ArmhandLogEntry (C: ArmhandLogEntry)                      | 45 |
| 9.3.33 HandErrorState (C: ArmhandHandErrorState)                 | 45 |

## 1.1 关于本手册

感谢您购买本公司的产品。

本手册记载了正确安装使用机器人的以下说明：

- 机器人二次开发接口 SDK(C++) 的使用。

安装使用该机器人系统前，请仔细阅读本手册与其他相关手册。

阅读之后，请妥善保管，以便随时取阅。

## 1.2 手册对象

本手册面向：

- 机器人应用开发工程师。

请务必保证以上人员具备基础的机器人操作、C++ 编程等所需的知识，并已接受本公司的相关培训。

## 1.3 版本记录

| 版本编号 | 日期        | 说明                          |
|------|-----------|-----------------------------|
| V1.0 | 2026.9.20 | 初始版本；支持臂手一体控制。              |
| V1.1 | 2026.9.26 | 更新 SetSoftLimit 接口和各关节活动范围。 |
| V1.2 | 2026.9.29 | 移除 MoveJPath 接口说明。          |

### 免责声明

感谢您购买本公司的 Synca XFS 一体化臂手总成（以下简称“本产品”）。为保障您的权益，请您在使用本产品前，**仔细阅读并充分理解本声明中的全部内容，特别是以粗体、下划线等显著方式提示的条款。**您开始使用本产品，即视为您已阅读、理解并同意接受本声明的约束。**若您不同意本声明中的任何内容，请勿使用本产品。**

如本声明的任何条款被认定无效、违法或不可执行，不影响其他条款的有效性与可执行性。

### 一、使用规范与责任承担

您应严格遵守随附的《用户手册》及相关法律法规，在安全的预期环境中安装和使用本产品。

您应对因您的下述行为（包括但不限于）所直接造成的任何损失、损害或责任自行负责：

1. 未按照《用户手册》的要求进行安装、使用、维护或存放；
2. 擅自对产品进行任何形式的改装、拆卸或修理；
3. 将产品用于非设计预期的用途或环境；
4. 将本产品与非我司生产、指定或认可的配件、零部件或耗材结合使用；
5. 其他明显疏忽或故意的不当行为。

在法律允许的最大范围内，对于因上述用户行为导致的任何损失或损害，杭州曦诺未来科技有限公司不承担责任。但本声明的任何内容均不排除或限制我司在如下方面的责任：（a）因我司产品存在《中华人民共和国产品质量法》等法律法规定义的缺陷而导致的人身伤亡或财产损失；（b）适用法律明确规定不得排除或限制的责任。

### 二、知识产权声明

**Xynova**

 **Xynova** 曦诺未来



以上标识是杭州曦诺未来科技有限公司的注册商标。本声明及产品包装上出现的其他产品名称、品牌标志等，均为其各自权利人所有的知识产权。

本产品及其相关技术方案受中华人民共和国专利、外观设计等知识产权法律的保护。本声明及随附《用户手册》的全部内容（包括但不限于文字、图表、图像）的版权归杭州曦诺未来科技有限公司。未经我司事先书面许可，任何单位或个人不得以任何形式（包括但不限于复制、翻印、翻译、改编等）对上述内容进行商业性使用或传播。

杭州曦诺未来科技有限公司

armhand SDK 是曦诺为臂手一体化机器人提供的二次开发接口库，在底层机械臂 SDK 与灵巧手 SDK 之上封装统一的 C++/C 接口 `xynova::armhandRobot`，对整机（机械臂 + 灵巧手）进行连接、使能、运动、力控、协同、固件与日志等控制与查询。

## 3.1 兼容性

### 3.1.1 控制器版本和机器人型号

控制器版本：xynova v3.2.0 及以后；灵巧手固件与驱动需匹配随包发布版本。

机器人型号：支持配套 xynova 系统的机械臂，配合曦诺灵巧手组成臂手一体化整机；不同机型可调用的接口有所差别。

### 3.1.2 运行环境与语言

- 操作系统：Linux（Ubuntu 20.04 / 22.04，x86\_64 / aarch64）。
- 语言绑定：C++（include/xynova/cpp/armhand）。
- 构建：CMake  $\geq 3.12$ 。

## 3.2 非实时控制

非实时模式向控制器下发带内部轨迹规划的运动指令，适用于点位运动、回零、标定、查询等对实时性要求不高的场景：

- 运动控制：关节空间绝对运动 `MoveAbsJ`、回零 `Home`、暂停 / 恢复 / 停止；
- 使能与模式：上下电使能 `EnableArm`、自动 / 手动模式 `SetOperatingMode`、拖动 `SetDragSpace`、速度与加速度设置；
- 运动学与标定：逆解 / 正解、关节零位标定、坐标系标定、软限位；
- 状态查询：关节状态、版本、设备信息、错误信息、控制器日志；
- 固件与备份：固件升级、控制器备份、NTP 对时。

### 3.3 实时控制

实时模式以固定周期（1 ms）与控制器交互，用于伺服跟随与高频闭环。以下 StartXXX 方法系统会进入实时模式，StopXXX 退出实时模式：

- 点位跟随：调用 StartFollowPosition 启动位置跟随，通过 UpdateFollowPosition 持续更新目标位置，结束时调用 StopFollowPosition。使用示例见 example/test\_follow\_joint.cpp。
- 伺服位置控制：调用 StartServo 启动伺服控制，通过 ServoJ 持续下发关节目标位置，结束时调用 StopServo。可配低通滤波 SetFilterLimit。使用示例见 example/test\_servoj.cpp。
- 关节阻抗控制：调用 StartJointImpedance 进入关节阻抗模式，通过 SetJointImpedance 设置或更新阻抗参数；控制结束后调用 StopJointImpedance。使用示例见 example/test\_joint\_impedance.cpp。对于需要周期性下发指令的控制任务，可通过 SetControlLoop 设置控制回调（回调周期为 1ms），再调用 StartLoop 启动控制循环。
- 关节力矩控制：调用 StartTorqueControl 进入关节力矩控制模式，通过 SendTorque 下发目标关节力矩；控制结束后调用 StopTorqueControl。使用示例见 example/test\_torque\_control.cpp。对于需要周期性下发指令的控制任务，可通过 SetControlLoop 设置控制回调（回调周期为 1ms），再调用 StartLoop 启动控制循环。
- 机器人状态获取：调用 UpdateRobotState 更新机器人状态，再调用 GetStateData 读取状态数据。状态数据的字段、单位及关节排列顺序见“9.2.4 RtStateField (C: ArmhandRtStateField)”一节。需要连续获取状态时，在控制循环中按规定周期执行上述操作，并检查接口返回值。
- 轴空间点动：使用 StartJog 启动指定关节的点动运动，使用 StopJog 停止点动。调用时应按接口参数指定关节、运动方向和速度；参数取值及使用示例见 StartJog 接口说明。使用示例见 example/test\_start\_jog.cpp。
- RT 回调循环：通过 SetControlLoop 设置控制回调，调用 StartLoop 启动循环，结束时调用 StopLoop 停止循环。该循环用于周期性执行控制逻辑；它与上述具体控制模式的启动、停止接口之间的调用关系，请参见示例 example/test\_rt\_joint\_position\_control.cpp。

本章介绍如何配置并运行一个 xynova SDK C++ 程序。

## 4.1 设置

### 4.1.1 硬件设置

关于机器人本体和控制柜等硬件的设置，请参考《xynova 控制系统使用手册 V3.2.1》。除了网络配置外，使用 xynova SDK 无需其他额外的硬件设置。

### 4.1.2 网络配置

xynova SDK 通过以太网（TCP/IP）连接机器人。通过有线或无线连接皆可，使用户 PC 和机器人连接同一局域网。如果只使用非实时控制，对于网络性能要求不高，可以通过无线连接。

使用实时控制的话推荐通过有线直连到机器人。机器人配置有 2 个网口，一个是外网口，一个是直联网口。直联网口默认静态 IP 地址是 192.168.1.160。连接机器人有两种方式：

- 连接方式 1：机器人与用户 PC 采用网线直连的方式连接。如果用户工控机与机器人不处于同一个网段，需要配置用户 PC 的 IP 使其与机器人静态 IP 地址处于同一个网段，例如 192.168.1.22。
- 连接方式 2：机器人外网口连接路由器或者交换机，用户 PC 也连接路由器或者交换机，两者处于同一局域网。

注：推荐使用方式 1 进行连接，连接方式 2 网络通信质量差时可能会造成机器人运动不稳定现象。

### 4.1.3 机器人功能设置

无需通过上位机进行任何设置，用户可直接用 xynova SDK 控制机器人。

## 4.2 编译

### 4.2.1 xynova SDK 工程包说明

```
armhand-sdk-< 版本>-< 架构>-< 日期>
├─ include: 头文件目录 (C++ 公开接口)
├─ lib: 库文件, 按 linux/< 架构>/ 组织 (libarmhand_sdk.a 及各底层库)
├─ example: 示例程序源码 (.cpp/.c/.hpp 及 example/CMakeLists.txt)
├─ bin: 预编译示例可执行程序 + 配置文件 + 绘图脚本
├─ scripts: 环境与依赖自检脚本 check_env.sh
├─ CMakeLists.txt: 包顶层构建脚本 (自动识别平台/架构并链接 lib/)
├─ build.sh: 一键编译示例
└─ README.txt: 发布说明与快速开始
```

### 4.2.2 C++ 编译

xynova SDK 发布包内的示例程序使用 CMake 构建 (CMake 版本不低于 3.12, 编译器需支持 C++14)。解压发布包后在根目录执行 **build.sh** 即可一键编译, 构建脚本会自动识别平台与架构并链接 lib/ 下的库, 无需手动配置头文件路径与链接库。

#### 4.2.2.1 Linux 平台编译

解压发布包后, 在包根目录执行 **build.sh** 即可编译全部示例, 产物位于 **build/example/**:

```
tar -xzf armhand-sdk-0.1.0-x86_64-< 日期>.tar.gz
cd armhand-sdk-0.1.0-x86_64-< 日期>
bash scripts/check_env.sh      # 可选: 检查库文件与运行依赖是否齐全
bash build.sh                  # 一键编译全部示例, 产物在 build/example/
```

## 4.3 本地配置文件

### 4.3.1 运行日志

运行 SDK 程序并连接到机器人之后, SDK 会在调用设置数据、执行操作等接口时打印日志到文件, 包含调用参数和返回结果等信息。可在遇到问题时将日志提供给曦诺技术支持人员, 以便分析排查。

日志默认的设置如下:

- 文件路径: <当前工作目录>/\_xynova\_log\_/<机器人uid>/xynovaSDK\_<YYYY-MM-DD>.log
- 最多保留 7 天的日志

目前支持用户自定义配置日志的输出位置和保留天数, 需要用户在执行目录下放置一个名为 **user\_log\_settings.json** 的文件, 文件结构如下:

```
{
  "_log_storage_directory_": "user_defined_log_storage_directory",
  "_retention_days_": 5
}
```

`_log_storage_directory_` 为用户指定的日志存储目录路径，替换默认日志存储目录 `_xynova_log_`，建议使用相对路径的方式写入；`_retention_days_` 为日志保留天数。

目前也提供了全局日志，全局日志的输出位置固定在运行程序所在目录下名为 `logs` 的文件夹，最多保留 7 天的全局日志。

### 4.3.2 实时模式等待超时时间设置

实时模式接收状态数据默认的等待超时时间是 1ms，根据现场实际的网络情况，可以在执行目录下，增加配置文件（`xynovasdk_config.json`）来设置超时时间，以适应实际情况和优化实时效果。若设备网络情况较好，并出现控制一段时间后，发送的运动指令有延迟执行的现象，可适当将超时时间调大，设置范围为 1~20ms，具体的 json 文件格式如下：

```
{
  "rt":{
    "_timeout_": 1
  }
}
```

## 4.4 语言

xynova SDK 的错误码信息和异常信息支持中文和英文，根据用户 PC 设置的系统语言而定。设置显示中文则返回中文错误信息，非中文则返回英文错误信息。

## 4.5 Linux 实时环境配置（可选）

xynova 控制器实时模式接收运动命令的周期为 1ms，客户端需保证发送频率至少为 1 kHz。如果计算量较大，推荐安装实时内核。

### 步骤 1：安装依赖

```
apt-get install build-essential bc curl ca-certificates fakeroot gnupg2 libssl-dev
↪ lsb-release libelf-dev bison flex cmake libeigen3-dev
```

### 步骤 2：下载实时内核补丁

通过 `uname -r` 命令可以知道本机正在使用的内核；

通过网站 <https://www.kernel.org/pub/linux/kernel/projects/rt/> 查找离现在内核版本最接近的 kernel；

下载文件：

```
curl -SLO https://www.kernel.org/pub/linux/kernel/v4.x/linux-4.14.12.tar.xz
curl -SLO https://www.kernel.org/pub/linux/kernel/v4.x/linux-4.14.12.tar.sign
curl -SLO https://www.kernel.org/pub/linux/kernel/projects/rt/4.14/older/patch-4.14.12-rt10
↪ .patch.xz
curl -SLO https://www.kernel.org/pub/linux/kernel/projects/rt/4.14/older/patch-4.14.12-rt10
↪ .patch.sign
```

如果国内网速很慢可以直接从网站上下载或者找其他镜像源；

解压：

```
xz -d linux-4.14.12.tar.xz
xz -d patch-4.14.12-rt10.patch.xz
```

检查 sign 文件完整性

```
gpg2 --verify linux-4.14.12.tar.sign
```

会得到类似于如下的信息：

```
gpg: Signature made Fr 05 Jan 2018 06:49:11 PST using RSA key ID 6092693E
gpg: Can't check signature: No public key
```

记下 ID 6092693E 执行：

```
gpg2 --keyserver hkp://keys.gnupg.net --recv-keys 0x6092693E
```

同理对于 patch 文件，执行相同的操作。

下载完成 server key 后再次验证，若得到如下信息就说明是正确的。

```
$ gpg2 --verify linux-4.14.12.tar.sign
gpg: assuming signed data in 'linux-4.14.12.tar'
gpg: Signature made Fr 05 Jan 2018 06:49:11 PST using RSA key ID 6092693E
gpg: Good signature from "Greg Kroah-Hartman <gregkh@linuxfoundation.org>" [unknown]
gpg: aka "Greg Kroah-Hartman <gregkh@kernel.org>" [unknown]
gpg: aka "Greg Kroah-Hartman (Linux kernel stable release signing key) <greg@kroah.com>"
↳ [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg: There is no indication that the signature belongs to the owner. Primary key
↳ fingerprint: 647F 2865 4894 E3BD 4571 99BE 38DB BDC8 6092 693E
```

同理验证一下 patch 文件。

### 步骤 3：编译内核

解压：

```
tar xf linux-4.14.12.tar
cd linux-4.14.12
patch -p1 < ../patch-4.14.12-rt10.patch
```

配置内核：

```
make oldconfig
```

出现以下信息：

```
Preemption Model
1. No Forced Preemption (Server) (PREEMPT_NONE)
2. Voluntary Kernel Preemption (Desktop) (PREEMPT_VOLUNTARY)
3. Preemptible Kernel (Low-Latency Desktop) (PREEMPT__LL) (NEW)
4. Preemptible Kernel (Basic RT) (PREEMPT_RT) (NEW)
> 5. Fully Preemptible Kernel (RT) (PREEMPT_RT_FULL) (NEW)
```

选 5 然后一直 enter。

开始编译：

```
fakeroot make -j4 deb-pkg
```

dpkg 安装：

```
sudo dpkg -i ../linux-headers-4.14.12-rt10_*.deb ../linux-image-4.14.12-rt10_*.deb
```

#### 步骤 4：验证是否安装成功

重启一下，进 ubuntu 高级选项，可以看到你安装的内核。选择新安装的内核进入后，通过 `uname -r` 查看对应内核版本，如果版本正确，`/sys/kernel/realtime` 里内容是 1。

本章描述对外发布的 SDK `xynova::armhandRobot`（臂手一体化门面），按**整机 Robot**、**机械臂 Arm**、**灵巧手 Hand** 三层对象模型组织。运行环境为 Linux，语言绑定为 C++。以下签名均取自公开头文件，为权威来源。

## 5.1 对象模型与通用约定

### 5.1.1 对象模型

| C++ 类 ( <code>xynova::armhandRobot</code> ) | 职责                                |
|---------------------------------------------|-----------------------------------|
| Robot                                       | 整机门面：连接、聚合状态、安全协调、固件、臂手协同；持有子系统   |
| Arm                                         | 机械臂子系统：使能 / 模式、运动、运动学、标定、力控、控制器日志 |
| Hand                                        | 灵巧手子系统：手指控制、设备信息、触觉 / 距离传感器       |

- 三个类均可独立构造；由 `Robot::Arm()` / `Robot::Hand()` 取得的是**借用引用**，生命周期不超过整机对象；类不可拷贝。
- 生命周期：构造 → Connect → 业务调用 → Disconnect；析构自动断开。
- 返回值：大多数操作接口返回 `ErrorCode`（成功 = `ErrorCode::Success / 0`）；数据经**引用出参**返回。`IsInMotion()`、`IsEnabled()`、`IsValid()` 和 `UpdateRobotState()` 返回 `bool`。具体返回类型以各接口签名为准；构造函数没有返回类型。
- 单位：角度 rad、笛卡尔位置 m、力 N、力矩  $N \cdot m$ 、速度 rad/s、温度 °C、电流 A；时间戳 `uint64_t`（Unix epoch 微秒）。
- 关节数 7（索引 0-6）；手指数上限 8（`ARMHAND_MAX_FINGERS`）；字符串 C ABI 为定长数组（短 64 / 中 128 / 长 256 字节），C++ 用 `std::string`。
- 回调运行在 **SDK 内部线程**，回调内禁止调用同一句柄的阻塞接口；`severity >= 4` 的 `ErrorInfo` 视为**故障级**。

## 5.2 Robot 整机接口

### 5.2.1 构造与生命周期

| 方法                | 签名                                  | 说明                                             |
|-------------------|-------------------------------------|------------------------------------------------|
| Robot             | Robot()                             | 以默认配置构造整机实例。                                   |
| Robot             | Robot(const RobotConfig &config)    | 以指定配置构造整机实例。                                   |
| ConstructionError | ErrorCode ConstructionError() const | 查询构造期错误码。                                      |
| IsValid           | bool IsValid() const                | 实例是否有效（句柄非空，等价于 ConstructionError()==Success）。 |

### 5.2.2 连接管理

| 方法                  | 签名                                                            | 说明                          |
|---------------------|---------------------------------------------------------------|-----------------------------|
| Connect             | ErrorCode Connect()                                           | 建立臂手一体化连接（臂与手一并连接并完成时序对齐）。  |
| Disconnect          | ErrorCode Disconnect()                                        | 断开臂手一体化连接，释放控制器资源。          |
| GetConnectionStatus | ErrorCode GetConnectionStatus(ConnectionStatus &status)       | 获取整机聚合连接状态（单枚举）。            |
| GetConnectionStatus | ErrorCode GetConnectionStatus(DeviceConnectionStatus &status) | 获取臂/手分端连接状态（可表达「臂通、手断」等组合）。 |

### 5.2.3 信息聚合

| 方法             | 签名                                                     | 说明                                           |
|----------------|--------------------------------------------------------|----------------------------------------------|
| GetVersionInfo | ErrorCode GetVersionInfo(VersionInfo &version)         | 查询整机版本（臂 SDK 版本 + 控制器固件 + 手 SDK/驱动固件聚合）。     |
| GetErrorInfo   | ErrorCode GetErrorInfo(std::vector<ErrorInfo> &errors) | 查询整机活动错误列表（臂硬件错误 + 软件 errorHandler 跟踪的接口错误）。 |
| GetState       | ErrorCode GetState(RobotState &state)                  | 查询整机状态快照（臂状态 + 手状态 + 同步状态 + 版本聚合）。           |
| GetDeviceInfo  | ErrorCode GetDeviceInfo(DeviceInfo &info)              | 查询整机设备信息（臂 + 手聚合到 DevicePartInfo）。           |
| GetErrorState  | ErrorCode GetErrorState(ErrorState &state)             | 查询整机错误状态（臂 + 手聚合）。                           |

### 5.2.4 整机安全协调

| 方法                      | 签名                                                                             | 说明                                         |
|-------------------------|--------------------------------------------------------------------------------|--------------------------------------------|
| Home                    | ErrorCode Home()                                                               | 整机回零：臂回零 + 手回零（手回零为张开的零位姿态）。               |
| ClearAllFaults          | ErrorCode ClearAllFaults()                                                     | 整机清障：清除臂/手历史遗留故障。                          |
| Reset                   | ErrorCode Reset()                                                              | 整机复位：臂复位 + 手回归安全状态。                        |
| RegisterFaultCallback   | ErrorCode RegisterFaultCallback(FaultCallback callback, uint64_t &callback_id) | 注册整机故障回调（ID 制，可多次注册）。                      |
| UnregisterFaultCallback | ErrorCode UnregisterFaultCallback(uint64_t callback_id)                        | 注销整机故障回调。                                  |
| GetSafetyStatus         | ErrorCode GetSafetyStatus(SafetyStatus &status)                                | 回读安全系统状态（链路在线/使能/上下电/操作模式/软限位/碰撞/急停/安全停止）。 |

### 5.2.5 日志管理

| 方法               | 签名                                                                       | 说明             |
|------------------|--------------------------------------------------------------------------|----------------|
| SetGlobalLogging | ErrorCode SetGlobalLogging(uint8_t enable)                               | 设置 SDK 全局日志开关。 |
| CleanupLogs      | ErrorCode CleanupLogs(LogCleanupMode mode, uint32_t retention_hours = 0) | 清理 SDK 自有日志目录。 |

### 5.2.6 固件管理

| 方法                | 签名                                                            | 说明                   |
|-------------------|---------------------------------------------------------------|----------------------|
| UpdateFirmware    | ErrorCode UpdateFirmware(const FirmwareUpdateConfig &config)  | 臂手协调固件升级（阻塞等待完成或超时）。 |
| GetUpdateProgress | ErrorCode GetUpdateProgress(FirmwareUpdateProgress &progress) | 查询固件升级进度。            |

### 5.2.7 臂手协同

| 方法                 | 签名                                                                                      | 说明                         |
|--------------------|-----------------------------------------------------------------------------------------|----------------------------|
| SynchronizeAction  | ErrorCode SynchronizeAction(const ArmTarget &arm_target, const HandTarget &hand_target) | 臂手同步动作（通用接口，覆盖抓取/松放/任意组合）。 |
| CalibrateJointZero | ErrorCode CalibrateJointZero()                                                          | 一键标定零点（会改写控制器机械零点，危险操作）。   |

### 5.2.8 运动学解算

| 方法          | 签名                                                                                                | 说明                          |
|-------------|---------------------------------------------------------------------------------------------------|-----------------------------|
| SetIKParams | ErrorCode SetIKParams(int max_iterations, double pos_tol, double rot_tol)                         | 设置数值逆解求解参数。                 |
| GetJointPos | ErrorCode GetJointPos(const Pose &cart_pose, const JointPosition &seed, JointPosition &joint_pos) | 数值逆解：法兰笛卡尔位姿 + 参考关节角 → 关节角。 |
| GetCartPose | ErrorCode GetCartPose(const JointPosition &joint_pos, Pose &cart_pose)                            | 数值正解：关节角 → 法兰笛卡尔位姿。         |

### 5.2.9 子系统访问器

| 方法   | 签名                                               | 说明                      |
|------|--------------------------------------------------|-------------------------|
| Arm  | ::xynova::armhandRobot::Arm &Arm()               | 获取臂子系统借用对象（非 const 重载）。 |
| Arm  | const ::xynova::armhandRobot::Arm &Arm() const   | 获取臂子系统借用对象（const 重载）。   |
| Hand | ::xynova::armhandRobot::Hand &Hand()             | 获取手子系统借用对象（非 const 重载）。 |
| Hand | const ::xynova::armhandRobot::Hand &Hand() const | 获取手子系统借用对象（const 重载）。   |

## 5.3 Arm 机械臂接口

### 5.3.1 构造与生命周期

| 方法                | 签名                                  | 说明                             |
|-------------------|-------------------------------------|--------------------------------|
| Arm               | Arm()                               | 创建独立机械臂实例（不连接，仅分配资源）。          |
| ConstructionError | ErrorCode ConstructionError() const | 构造是否成功（失败时后续方法返回 SysInternal）。 |
| IsValid           | bool IsValid() const                | 实例是否有效（句柄非空且构造成功）。             |

### 5.3.2 连接管理

| 方法                   | 签名                                                                                                                                                                                                                                | 说明                                                     |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Connect              | <code>ErrorCode Connect(const std::string &amp;controller_ip, uint16_t port = 8080, const std::string &amp;auth_key = std::string(), double connection_timeout_sec = 5.0, const std::string &amp;local_ip = std::string())</code> | 连接机械臂控制器。                                              |
| Disconnect           | <code>ErrorCode Disconnect()</code>                                                                                                                                                                                               | 断开机械臂连接，释放控制器侧资源。                                      |
| GetConnectionStatus  | <code>ErrorCode GetConnectionStatus(ConnectionStatus &amp;status)</code>                                                                                                                                                          | 获取臂当前连接状态（已连接 / 断开 / 错误）。                              |
| SetConnectionHandler | <code>ErrorCode SetConnectionHandler(ConnectionCallbackCpp callback)</code>                                                                                                                                                       | 注册连接状态变化回调（断线重连/故障告警场景），传空 <code>function</code> 取消订阅。 |

### 5.3.3 状态查询

| 方法                        | 签名                                                                                                                                                  | 说明                                                |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| GetJointStates            | <code>ErrorCode GetJointStates(std::vector&lt;JointState&gt; &amp;joints)</code>                                                                    | 查询 7 关节完整状态（位置/速度/温度/电流等）。                        |
| GetBaseFrame              | <code>ErrorCode GetBaseFrame(WorldFrame &amp;frame)</code>                                                                                          | 查询当前基坐标系（相对世界坐标系的位姿与名称）。                          |
| GetOperatingMode          | <code>ErrorCode GetOperatingMode(OperatingMode &amp;mode)</code>                                                                                    | 查询操作模式（Manual 手动 / Automatic 自动）。                 |
| GetCollisionState         | <code>ErrorCode GetCollisionState(CollisionState &amp;state)</code>                                                                                 | 查询当前碰撞状态（正常 / 碰撞中）。                               |
| EnableCollisionDetection  | <code>ErrorCode EnableCollisionDetection(const std::array&lt;double, 7&gt; &amp;sensitivity, StopLevel behaviour, double fallbackCompliance)</code> | 打开碰撞检测。                                           |
| DisableCollisionDetection | <code>ErrorCode DisableCollisionDetection()</code>                                                                                                  | 关闭碰撞检测。                                           |
| SetCollisionHandler       | <code>ErrorCode SetCollisionHandler(CollisionCallbackCpp callback)</code>                                                                           | 订阅碰撞事件（SDK 独立线程回调）；传空 <code>function</code> 取消订阅。 |
| GetErrorInfo              | <code>ErrorCode GetErrorInfo(std::vector&lt;ErrorInfo&gt; &amp;errors)</code>                                                                       | 查询臂端当前活动错误列表（硬件错误 + SDK 软件错误）。                    |
| GetVersionInfo            | <code>ErrorCode GetVersionInfo(VersionInfo &amp;version)</code>                                                                                     | 查询臂端版本信息（控制器软件/硬件版本、SDK 版本等）。                     |
| GetDeviceInfo             | <code>ErrorCode GetDeviceInfo(DevicePartInfo &amp;info)</code>                                                                                      | 查询臂端设备信息（产品型号 / 序列号 / 硬件版本 / 软件版本）。               |

### 5.3.4 固件 / 备份 / 时钟

| 方法                     | 签名                                                                                                                               | 说明                                       |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| UpdateFirmware         | <code>ErrorCode UpdateFirmware(const std::string &amp;local_path)</code>                                                         | 升级控制器固件（阻塞等待完成或超时）。                      |
| ExportControllerBackup | <code>ErrorCode ExportControllerBackup(const std::string &amp;local_path, const std::vector&lt;BackupItem&gt; &amp;items)</code> | 导出控制器备份，可指定多项内容（日志/RL 程序/配置等），阻塞等待完成或超时。 |
| SyncControllerTime     | <code>ErrorCode SyncControllerTime(const std::string &amp;ntp_server)</code>                                                     | 配置 NTP 服务器并同步控制器时钟到该服务器。                 |

### 5.3.5 模式与使能

| 方法                 | 签名                                                           | 说明                                |
|--------------------|--------------------------------------------------------------|-----------------------------------|
| EnableArm          | <code>ErrorCode EnableArm(bool enabled)</code>               | 臂上电/下电。                           |
| SetOperatingMode   | <code>ErrorCode SetOperatingMode(OperatingMode mode)</code>  | 设置操作模式（Manual 手动 / Automatic 自动）。 |
| SetDragSpace       | <code>ErrorCode SetDragSpace(DragSpaceMode mode)</code>      | 设置拖动空间模式（须先切到 Manual 模式）。         |
| DisableDrag        | <code>ErrorCode DisableDrag()</code>                         | 退出拖动模式。                           |
| SetAcceleration    | <code>ErrorCode SetAcceleration(double accel_percent)</code> | 设置全局加速度百分比（在线生效）。                 |
| CalibrateJointZero | <code>ErrorCode CalibrateJointZero()</code>                  | 一键标定关节零点（会改写控制器机械零点，危险操作）。        |

### 5.3.6 运动控制

| 方法               | 签名                                                                    | 说明                           |
|------------------|-----------------------------------------------------------------------|------------------------------|
| StartMoveAbsJ    | <code>ErrorCode StartMoveAbsJ(const JointPosition &amp;target)</code> | 非阻塞启动关节绝对运动下发后立即返回。          |
| MoveAbsJ         | <code>ErrorCode MoveAbsJ(const JointPosition &amp;target)</code>      | 关节空间绝对运动（阻塞到运动完成，含超时保护）。     |
| ClearMotionQueue | <code>ErrorCode ClearMotionQueue()</code>                             | 清空运动指令队列（不影响正在执行的指令）。        |
| StopMotion       | <code>ErrorCode StopMotion()</code>                                   | 停止运动（立即停止当前运动并清空指令队列）。       |
| PauseMotion      | <code>ErrorCode PauseMotion()</code>                                  | 暂停运动（保留队列，可 Resume 恢复）。      |
| ResumeMotion     | <code>ErrorCode ResumeMotion()</code>                                 | 恢复被 Pause 暂停的运动。             |
| Home             | <code>ErrorCode Home()</code>                                         | 臂回零（运动到机械零点）。                |
| ClearArmFaults   | <code>ErrorCode ClearArmFaults()</code>                               | 清除臂端故障（错误状态、运动停止状态等）。        |
| Reset            | <code>ErrorCode Reset()</code>                                        | 臂复位（模式/状态恢复到默认值，常用于异常后恢复）。   |
| IsInMotion       | <code>bool IsInMotion() const</code>                                  | 查询臂是否运动中（异步运动启动后到完成前为 true）。 |

| 方法              | 签名                                            | 说明                                         |
|-----------------|-----------------------------------------------|--------------------------------------------|
| IsEnabled       | bool IsEnabled() const                        | 查询臂是否已上电使能。                                |
| GetMotionState  | ErrorCode GetMotionState(MotionState &state)  | 查询臂运动状态机状态 (idle/moving/paused/stopped 等)。 |
| GetRtActiveMode | ErrorCode GetRtActiveMode(RtActiveMode &mode) | 查询当前实时控制子模式 (ServoJ / torque / follow 之一)。 |

### 5.3.7 运动学 / 标定 / 软限位

| 方法                | 签名                                                                                                | 说明                          |
|-------------------|---------------------------------------------------------------------------------------------------|-----------------------------|
| GetJointPos       | ErrorCode GetJointPos(const Pose &cart_pose, const JointPosition &seed, JointPosition &joint_pos) | 数值逆解：法兰笛卡尔位姿 + 参考关节角 → 关节角。 |
| GetCartPose       | ErrorCode GetCartPose(const JointPosition &joint_pos, Pose &cart_pose)                            | 数值正解：关节角 → 法兰笛卡尔位姿。         |
| AdjustSpeedOnline | ErrorCode AdjustSpeedOnline(double scale)                                                         | 在线调速（运动中按比例缩放速度，可动态调整）。     |
| GetSoftLimit      | ErrorCode GetSoftLimit(SoftLimitConfig &config)                                                   | 查询软限位配置（各关节最大/最小角度限制）。      |
| SetSoftLimit      | ErrorCode SetSoftLimit(const SoftLimitConfig &config)                                             | 设置软限位配置（生效后限制运动指令的关节范围）。    |
| RecoverFromEstop  | ErrorCode RecoverFromEstop()                                                                      | 急停恢复（清除急停状态后允许重新上电/运动）。     |

### 5.3.8 基坐标系 / 控制器日志 / 系统

| 方法                     | 签名                                                                                                                                         | 说明                        |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| SetBaseFrame           | ErrorCode SetBaseFrame(const WorldFrame &frame)                                                                                            | 设置基坐标系（相对世界坐标系），重启控制器后生效。 |
| QueryControllerLog     | ErrorCode QueryControllerLog(uint8_t max_count, const std::vector<LogLevel> &levels, std::vector<LogEntry> &out_logs, uint32_t offset = 0) | 查询控制器日志（带分页与等级过滤）。        |
| RebootControllerSystem | ErrorCode RebootControllerSystem()                                                                                                         | 重启控制器系统（阻塞等待重启完成或超时）。     |

### 5.3.9 状态 / 急停恢复

| 方法                | 签名                                                  | 说明                                                 |
|-------------------|-----------------------------------------------------|----------------------------------------------------|
| GetOperationState | ErrorCode GetOperationState (OperationState &state) | 查询控制器当前运行状态 (idle/running/error 等)。                |
| GetPowerState     | ErrorCode GetPowerState (PowerState &state)         | 查询上下电及急停状态 (含 servoOn/servoOff/emergencyStop 等组合)。 |
| RecoverState      | ErrorCode RecoverState (RecoverOption option)       | 急停/安全停止恢复 (按选项恢复)。                                 |

### 5.3.10 实时轨迹跟随

| 方法                   | 签名                                                                           | 说明                                   |
|----------------------|------------------------------------------------------------------------------|--------------------------------------|
| StartFollowPosition  | ErrorCode StartFollowPosition (const JointPosition &start_jnt, double scale) | 启动实时轨迹跟随模式 (给定起始关节与缩放比例)。            |
| UpdateFollowPosition | ErrorCode UpdateFollowPosition (const JointPosition &target_jnt)             | 更新实时轨迹跟随目标 (须先 StartFollowPosition)。 |
| StopFollowPosition   | ErrorCode StopFollowPosition ()                                              | 停止实时轨迹跟随模式并退出 RT 子模式。                |

### 5.3.11 ServoJ: 实时轴空间伺服位置控制

| 方法         | 签名                                                               | 说明                                    |
|------------|------------------------------------------------------------------|---------------------------------------|
| StartServo | ErrorCode StartServo (double cycle, double lookahead, double kp) | 进入实时轴空间伺服模式 (ServoJ)。                 |
| ServoJ     | ErrorCode ServoJ (const JointPosition &target)                   | 发送关节位置伺服指令 (须先 StartServo)。           |
| StopServo  | ErrorCode StopServo ()                                           | 退出实时伺服模式 (stopServoJoint + stopMove)。 |

### 5.3.12 力传感器标定 / 力控参数 / 默认速度

| 方法                   | 签名                                                                | 说明                                                             |
|----------------------|-------------------------------------------------------------------|----------------------------------------------------------------|
| CalibrateForceSensor | ErrorCode CalibrateForceSensor (bool allAxes, uint8_t jointIndex) | 标定力传感器/力矩传感器零点。                                                |
| SetFcGain            | ErrorCode SetFcGain (const std::array<double, 7> &gain)           | 设置力控带宽 (forceControl().setFcGain, 须在 StartTorqueControl 之前调用)。 |
| SetFriction          | ErrorCode SetFriction (const std::array<double, 7> &friction)     | 设置摩擦力补偿系数 (forceControl().setFriction)。                        |
| SetDefaultSpeed      | ErrorCode SetDefaultSpeed (double speedPercent)                   | 设置默认关节速度百分比 (缓存到 motionParams, 下次运动时下发到控制器)。                   |

| 方法             | 签名                                      | 说明                                                 |
|----------------|-----------------------------------------|----------------------------------------------------|
| SetDefaultZone | ErrorCode SetDefaultZone(double zoneMm) | 设置默认转弯区（缓存到 motionParams.blendRadius，下次运动时下发到控制器）。 |

### 5.3.13 RT 网络容忍度 / 状态反馈

| 方法                    | 签名                                                                   | 说明                                          |
|-----------------------|----------------------------------------------------------------------|---------------------------------------------|
| SetRtNetworkTolerance | ErrorCode SetRtNetworkTolerance(int toleranceMs)                     | 设置实时通信容忍延迟（调用 RT 接口前可设置）。                   |
| UpdateRobotState      | bool UpdateRobotState(int timeoutMs)                                 | 更新 RT 状态数据缓存（阻塞等待下一帧或超时）。                   |
| GetStateData          | ErrorCode GetStateData(RtStateField field, std::vector<double> &out) | 读取最新一帧指定字段的状态数据（配合 UpdateRobotState 抽帧后读取）。 |

### 5.3.14 ServoJ 滤波 / RT 回调循环

| 方法             | 签名                                                                                                                                | 说明                                                 |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| SetFilterLimit | ErrorCode SetFilterLimit(bool enable, int frequency)                                                                              | 启用/禁用 ServoJ 指令低通滤波（须在 StartServo 之后、ServoJ 之前调用）。 |
| SetControlLoop | ErrorCode SetControlLoop(const std::function<std::vector<double>()> &callback, int priority = 0, bool useStateDataInLoop = false) | 注册 RT 回调循环函数。                                      |
| StartLoop      | ErrorCode StartLoop(bool blocking)                                                                                                | 启动 RT 回调循环（须先 SetControlLoop）。                     |
| StopLoop       | ErrorCode StopLoop()                                                                                                              | 停止 RT 回调循环（异步停止，回调返回空后退出）。                         |

### 5.3.15 轴空间阻抗控制

| 方法                  | 签名                                                                    | 说明                                                                                                                                   |
|---------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| StartJointImpedance | ErrorCode StartJointImpedance(const std::array<double, 7> &stiffness) | 进入关节阻抗控制模式（RtCommand + jointImpedance + 设置阻抗系数）。                                                                                     |
| StopJointImpedance  | ErrorCode StopJointImpedance()                                        | 退出关节阻抗控制模式。                                                                                                                          |
| SetJointImpedance   | ErrorCode SetJointImpedance(const std::array<double, 7> &stiffness)   | 设置关节阻抗参数（须在 StartJointImpedance 之后调用，仅动态更新阻抗系数）。阻抗模式的目标关节位置请改用 SetControlLoop 回调按 1 ms 周期下发（推荐，参见 example/test_joint_impedance.cpp）。 |

### 5.3.16 轴空间力矩控制

| 方法                 | 签名                                                           | 说明                                                                                                         |
|--------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| StartTorqueControl | ErrorCode StartTorqueControl(double filter_frequency = 50.0) | 进入轴空间力矩控制模式。                                                                                               |
| SendTorque         | ErrorCode SendTorque(const std::array<double, 7> &torque)    | 发送关节力矩指令（须先 StartTorqueControl）。推荐改用 SetControlLoop 回调按 1 ms 周期持续下发力矩（参见 example/test_torque_control.cpp）。 |
| StopTorqueControl  | ErrorCode StopTorqueControl()                                | 退出轴空间力矩控制模式。                                                                                               |

### 5.3.17 轴空间 Jog

| 方法       | 签名                                                                                | 说明                                                           |
|----------|-----------------------------------------------------------------------------------|--------------------------------------------------------------|
| StartJog | ErrorCode StartJog(double rate, double step, uint8_t joint_index, bool direction) | 开始轴空间 jog 运动（须先 SetOperatingMode(Manual) + EnableArm(true)）。 |
| StopJog  | ErrorCode StopJog()                                                               | 停止 jog 运动。                                                   |

## 5.4 Hand 灵巧手接口

### 5.4.1 构造与生命周期

| 方法                | 签名                                  | 说明                    |
|-------------------|-------------------------------------|-----------------------|
| Hand              | Hand()                              | 创建独立灵巧手实例（不连接，仅分配资源）。 |
| ConstructionError | ErrorCode ConstructionError() const | 查询构造是否成功。             |
| IsValid           | bool IsValid() const                | 实例是否有效（句柄非空且构造成功）。    |

### 5.4.2 连接管理

| 方法                  | 签名                                                                                                                                                                                | 说明               |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| Connect             | ErrorCode Connect(const std::string &can_type, const std::string &can_interface, uint32_t can_id, const std::string &device_name, const HandSchedulingToggles *toggles = nullptr) | 连接灵巧手。           |
| Disconnect          | ErrorCode Disconnect()                                                                                                                                                            | 断开灵巧手连接（自动去使能）。  |
| GetConnectionStatus | ErrorCode GetConnectionStatus(ConnectionStatus &status)                                                                                                                           | 获取当前连接状态（不触发通讯）。 |

### 5.4.3 设备信息

| 方法            | 签名                                            | 说明                                          |
|---------------|-----------------------------------------------|---------------------------------------------|
| GetDeviceInfo | ErrorCode GetDeviceInfo(DevicePartInfo &info) | 查询手端设备信息（聚合产品型号 / 序列号 / 硬件版本 / 软件版本 / 手类型）。 |

### 5.4.4 状态

| 方法              | 签名                                                           | 说明                                                  |
|-----------------|--------------------------------------------------------------|-----------------------------------------------------|
| GetFingerStates | ErrorCode GetFingerStates(std::vector<FingerState> &fingers) | 查询全部手指状态，固定 6 指槽位                                   |
| GetErrorState   | ErrorCode GetErrorState(HandErrorState &state)               | 查询手端原始错误状态：全局 warning/fault 位图 + 按关节 ID-1 索引的错误码数组。 |

### 5.4.5 使能 / 运动控制

| 方法                   | 签名                                                                       | 说明                          |
|----------------------|--------------------------------------------------------------------------|-----------------------------|
| Enable               | ErrorCode Enable(bool enabled)                                           | 使能/去使能灵巧手。                  |
| SetControlMode       | ErrorCode SetControlMode(HandControlMode mode)                           | 设置手指控制模式（位置/力）。             |
| SetFingerAngles      | ErrorCode SetFingerAngles(const std::vector<double> &positions_rad)      | 同步下发手指目标角度（rad），阻塞等待到位。     |
| SetFingerAnglesAsync | ErrorCode SetFingerAnglesAsync(const std::vector<double> &positions_rad) | 异步下发手指目标角度（rad），立即返回，不等待到位。 |

### 5.4.6 传感器使能

| 方法                  | 签名                                         | 说明                                                     |
|---------------------|--------------------------------------------|--------------------------------------------------------|
| EnableTactileWrench | ErrorCode EnableTactileWrench(bool enable) | 使能/去使能触觉 wrench（指尖六维力 [Fx,Fy,Fz,Tx,Ty,Tz]）数据订阅（运行时切换）。 |
| EnableTactileTaxel  | ErrorCode EnableTactileTaxel(bool enable)  | 使能/去使能触觉 taxel 阵列（每个触觉点的六维力）周期订阅。                      |
| EnableRange         | ErrorCode EnableRange(bool enable)         | 使能/去使能接近觉（DToF 距离图）周期订阅。                               |

### 5.4.7 传感器读取

| 方法               | 签名                                                                                            | 说明                                                       |
|------------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------|
| GetTactileWrench | ErrorCode GetTactileWrench(std::vector<TactileWrench> &out, uint64_t *timestamp_us = nullptr) | 读取触觉 wrench 数据快照。                                        |
| GetTactileTaxel  | ErrorCode GetTactileTaxel(std::vector<TactileTaxel> &out, uint64_t *timestamp_us = nullptr)   | 读取触觉 taxel 阵列数据快照。                                       |
| GetRange         | ErrorCode GetRange(RangeMeasurement &out, uint64_t *timestamp_us = nullptr)                   | 读取接近觉距离图快照 (width × height 个点, 行优先 index = y*width + x)。 |

### 5.4.8 故障恢复

| 方法        | 签名                     | 说明                            |
|-----------|------------------------|-------------------------------|
| Recover   | ErrorCode Recover()    | 故障恢复 (适用于通讯中断等场景, 恢复后重建控制状态)。 |
| IsEnabled | bool IsEnabled() const | 查询手是否使能 (不触发通讯)。              |

## 5.5 错误码

### 5.5.1 形态

```
using namespace xynova::armhandRobot;
ErrorCode ec = robot.Connect();
if (ec != ErrorCode::Success) {
    printf("%s\n", ToString(ec)); // 可读描述字符串 (如 "Connection failed")
    int32_t v = static_cast<int32_t>(ec); // 数值
}
```

接口返回异常可通过错误码查看。错误码表如下, ToString(ec) 返回人类可读字符串 (如"Connection failed")。

### 5.5.2 错误码全表

| 名称                   | 值          | 说明         |
|----------------------|------------|------------|
| Success              | 0          | 成功         |
| <b>通讯类</b>           |            |            |
| CommConnectionFailed | 0x01000001 | 连接失败       |
| CommTimeout          | 0x01000002 | 通讯超时       |
| CommAuthFailed       | 0x01000003 | 设备认证失败     |
| CommHandshakeFailed  | 0x01000004 | 加密握手失败     |
| CommDisconnected     | 0x01000005 | 当前未连接      |
| CommEstopPressed     | 0x01000006 | 急停按钮被按下    |
| CommModeMismatch     | 0x01000007 | 手动/自动模式不匹配 |

| 名称                                | 值          | 说明              |
|-----------------------------------|------------|-----------------|
| CommPowerStateInvalid             | 0x01000008 | 上下电状态不允许        |
| CommRobotNotIdle                  | 0x01000009 | 机器人非空闲          |
| CommControlModeMismatch           | 0x0100000A | 控制模式不匹配         |
| CommRobotInMotion                 | 0x0100000B | 机器人运动中          |
| CommUdpError                      | 0x0100000C | UDP 通讯错误        |
| CommLocalIpInvalid                | 0x0100000D | 本机 IP 配置无效      |
| CommNotAuthorized                 | 0x0100000E | SDK 未授权         |
| CommVersionIncompatible           | 0x0100000F | 版本不兼容           |
| <b>参数类</b>                        |            |                 |
| ParamInvalid                      | 0x02000001 | 参数无效            |
| ParamOutOfRange                   | 0x02000002 | 参数超出允许范围        |
| ParamMissing                      | 0x02000003 | 必需参数未配置         |
| ParamIkFailed                     | 0x02000004 | 逆解错误            |
| ParamIkSingularity                | 0x02000005 | 逆解奇异点           |
| ParamIkSoftLimit                  | 0x02000006 | 逆解超出软限位         |
| ParamIkOutOfRange                 | 0x02000007 | 目标点超出运动范围       |
| ParamIkStepTooLarge               | 0x02000008 | 单步距离过大          |
| ParamConfDataError                | 0x02000009 | 配置数据错误          |
| ParamTrajectoryUnreachable        | 0x0200000A | 轨迹不可达           |
| ParamConfNoSolution               | 0x0200000B | 指定 conf 参数下无解   |
| ParamCartPathConfMismatch         | 0x0200000C | 笛卡尔路径终点 conf 不符 |
| ParamTrajectoryThroughSingularity | 0x0200000D | 轨迹穿越奇异点         |
| ParamJointOutOfRange              | 0x0200000E | 轴角度超出运动范围       |
| ParamLockAxisDeviation            | 0x0200000F | 锁轴角度偏离          |
| ParamLockAxisUnreachable          | 0x02000010 | 锁轴状态无法到达        |
| ParamDataNotFound                 | 0x02000011 | 数据/指令名称未找到      |
| ParamDataNotReadable              | 0x02000012 | 数据不可读           |
| ParamDataNotWritable              | 0x02000013 | 数据不可写           |
| ParamDataValueError               | 0x02000014 | 数据值错误           |
| <b>状态类</b>                        |            |                 |
| StateNotEnabled                   | 0x03000001 | 设备未使能           |
| StateInMotion                     | 0x03000002 | 设备运动中           |
| StateWrongMode                    | 0x03000003 | 当前模式与请求操作不匹配    |
| StateNotAligned                   | 0x03000004 | 时序未对齐           |
| StateRtNetworkError               | 0x03000005 | 实时模式网络错误        |
| StateRtAlreadyMoving              | 0x03000006 | 已开始运动           |
| StateRtSingularity                | 0x03000007 | 起始位置为奇异点        |
| StateRtParamError                 | 0x03000008 | 实时模式参数错误        |
| StateRtCollisionStop              | 0x03000009 | 机器人处于碰撞停止       |
| StateRtRequestRejected            | 0x0300000A | 请求被拒绝           |
| StateRtEstop                      | 0x0300000B | 机器人处于急停状态       |
| StateRtNotStarted                 | 0x0300000C | 未调用开始运动         |
| <b>力控类</b>                        |            |                 |
| StateFcNotInitialized             | 0x03000101 | 力控未初始化          |
| StateFcInProgress                 | 0x03000102 | 力控指令未完成         |
| StateFcModeMismatch               | 0x03000103 | 非阻抗控制模式         |
| StateFcSearchRunning              | 0x03000104 | 搜索运动在运行中        |
| StateFcNotCartesianImpedance      | 0x03000105 | 非笛卡尔阻抗模式        |

| 名称                               | 值          | 说明          |
|----------------------------------|------------|-------------|
| StateFcSearchNotStarted          | 0x03000106 | 搜索运动未开始     |
| StateFcStiffnessFailed           | 0x03000107 | 阻抗刚度设置失败    |
| StateFcPowerOffDuringFc          | 0x03000108 | 力控过程中下电     |
| StateFcCmdTooFrequent            | 0x03000109 | 力控指令发送太频繁   |
| StateFcNeedRestart               | 0x0300010A | 需重新开始力控     |
| StateFcNotAllowed                | 0x0300010B | 力控模式下不允许的操作 |
| StateFcModuleError               | 0x0300010C | 力控模块处于错误状态  |
| StateFcTcpTooLong                | 0x0300010D | TCP 长度超限    |
| <b>硬件类</b>                       |            |             |
| HwJointFault                     | 0x04000001 | 关节故障        |
| HwCollision                      | 0x04000002 | 碰撞检测触发      |
| HwOverTemperature                | 0x04000003 | 关节过温        |
| HwCanError                       | 0x04000004 | CAN 总线错误    |
| HwJointPositionLimit             | 0x04000005 | 实际轴角度超限     |
| HwCartesianPositionLimit         | 0x04000006 | 实际末端位姿超限    |
| HwJointVelocityLimit             | 0x04000007 | 实际轴速度超限     |
| HwJointAccelerationLimit         | 0x04000008 | 实际轴加速度超限    |
| HwCommandTorqueLimit             | 0x04000009 | 指令力矩超限      |
| HwCommandTorqueDiscontinuity     | 0x0400000A | 指令力矩不连续     |
| HwSingularity                    | 0x0400000B | 机器人奇异       |
| HwInstability                    | 0x0400000C | 检测到不稳定      |
| HwElbowLimitViolation            | 0x0400000D | 臂角超限        |
| HwCartesianVelocityLimit         | 0x0400000E | 实际末端速度超限    |
| HwCartesianAccelerationLimit     | 0x0400000F | 实际末端加速度超限   |
| HwJointAccelerationDiscontinuity | 0x04000010 | 指令轴加速度不连续   |
| HwJointTorqueOverLimit           | 0x04000011 | 关节力矩超限      |
| HwSafetyZoneConflict             | 0x04000012 | 安全区域冲突      |
| <b>固件类</b>                       |            |             |
| FwIncompatible                   | 0x05000001 | 固件版本不兼容     |
| FwUpdateFailed                   | 0x05000002 | 固件升级失败      |
| FwChecksumMismatch               | 0x05000003 | 固件校验和不匹配    |
| FwTrajectoryTooShort             | 0x05000004 | 录制路径过短      |
| FwTrajectorySoftLimit            | 0x05000005 | 录制路径超软限位    |
| FwNtpSyncFailed                  | 0x05000006 | NTP 时间同步失败  |
| FwLoadOverLimit                  | 0x05000007 | 负载超额定       |
| FwLoadInfoError                  | 0x05000008 | 负载信息错误      |
| FwXCoreVersionTooLow             | 0x05000009 | xCore 版本过低  |
| FwInstanceTypeMismatch           | 0x0500000A | 实例类型与机型不符   |
| <b>同步/轨迹类</b>                    |            |             |
| SyncTimeout                      | 0x06000001 | 臂手同步等待超时    |
| SyncDriftExceeded                | 0x06000002 | 时序漂移超阈值     |
| SyncTrajectoryAngleTooSmall      | 0x06000003 | 轨迹夹角过小      |
| SyncTrajectoryControllerError    | 0x06000004 | 控制器状态错误     |
| SyncTrajectoryTooClose           | 0x06000005 | 相邻目标点过近     |
| SyncArcGenerationFailed          | 0x06000006 | 圆弧生成失败      |
| SyncArcStartTargetTooClose       | 0x06000007 | 圆弧起点终点过近    |
| SyncArcStartAuxTooClose          | 0x06000008 | 圆弧起点辅助点过近   |
| SyncArcAuxTargetTooClose         | 0x06000009 | 圆弧辅助点终点过近   |

| 名称                             | 值          | 说明              |
|--------------------------------|------------|-----------------|
| SyncArcAllPointsTooClose       | 0x0600000A | 圆弧三点过近          |
| SyncArcCollinear               | 0x0600000B | 圆弧三点共线          |
| SyncArcRadiusTooSmall          | 0x0600000C | 圆弧半径过小          |
| SyncArcAngleTooSmall           | 0x0600000D | 圆弧夹角过小          |
| SyncTrajShortAutoBlend         | 0x0600000E | 轨迹短于最小转弯区       |
| SyncNotSupportSingularityAvoid | 0x0600000F | 不支持奇异规避         |
| <b>系统类</b>                     |            |                 |
| SysInternal                    | 0x07000001 | SDK 内部错误        |
| SysNotSupported                | 0x07000002 | 当前不支持该功能        |
| SysAlreadyInitialized          | 0x07000003 | 已初始化            |
| SysNotInitialized              | 0x07000004 | 未初始化            |
| SysNotReceivingData            | 0x07000005 | 未开始接收数据         |
| SysBusy                        | 0x07000006 | 资源忙             |
| SysConfigError                 | 0x07000007 | 配置错误            |
| SysSubsystemError              | 0x07000008 | 子系统内部错误（日志/从站等） |
| <b>灵巧手专属类</b>                  |            |                 |
| HandSetModeFailed              | 0x08000001 | 设置手模式失败         |
| HandSetEnablementFailed        | 0x08000002 | 设置系统使能失败        |
| HandSetBackToZeroFailed        | 0x08000003 | 手回零失败           |
| HandClearErrorsFailed          | 0x08000004 | 清除手错误失败         |
| HandDataOutOfDate              | 0x08000005 | 手数据过期           |
| HandNoDataUpdated              | 0x08000006 | 手数据未更新          |
| HandCreateSlaveFailed          | 0x08000007 | 创建手从站失败         |
| HandProtocolStackInitFailed    | 0x08000008 | 手协议栈初始化失败       |

### 5.5.3 错误来源 ErrorSource

| 名称    | 值    | 含义       |
|-------|------|----------|
| Arm   | 0x01 | 机械臂      |
| Hand  | 0x02 | 灵巧手      |
| Robot | 0x03 | 臂手协同（整机） |

## 6.1 实时指令

用户下发的运动指令需要满足建议条件和必要条件。建议条件是为了让机器人运动更加平稳，性能最优。必要条件则是必须满足的，若不满足，机器人将会停止。

### 6.1.1 轴空间运动

#### 必要条件

轴空间轨迹平滑，至少保证速度是连续可导的：

$$\begin{aligned} q_{\min} &< q_c < q_{\max} \\ -\dot{q}_{\max} &< \dot{q}_c < \dot{q}_{\max} \\ -\ddot{q}_{\max} &< \ddot{q}_c < \ddot{q}_{\max} \\ -\dddot{q}_{\max} &< \dddot{q}_c < \dddot{q}_{\max} \end{aligned}$$

#### 建议条件

力矩指令不超限：

$$\begin{aligned} -\tau_{j\max} &< \tau_{jc} < \tau_{j\max} \\ -\dot{\tau}_{j\max} &< \dot{\tau}_{jc} < \dot{\tau}_{j\max} \end{aligned}$$

运动开始时：

$$\begin{aligned} q &= q_c \\ \dot{q}_c &= 0 \\ \ddot{q}_c &= 0 \end{aligned}$$

运动结束时：

$$\begin{aligned} \dot{q}_c &= 0 \\ \ddot{q}_c &= 0 \end{aligned}$$

### 6.1.2 力矩直接控制

#### 必要条件

力矩指令不超限且连续：

$$-\dot{\tau}_{jmax} < \dot{\tau}_{jc} < \dot{\tau}_{jmax}$$

$$-\tau_{jmax} < \tau_{jc} < \tau_{jmax}$$

### 6.1.3 运动限制条件

Synca XFS 轴空间运动限制条件：

| 参数   | 一轴   | 二轴   | 三轴   | 四轴  | 五轴   | 六轴<br>(WR1)                                                       | 七轴<br>(WR2)                                                       | 单位 |
|------|------|------|------|-----|------|-------------------------------------------------------------------|-------------------------------------------------------------------|----|
| 限位上限 | 178  | 120  | 178  | 145 | 178  | 详情参考<br>《Synca XFS<br>一体化臂手<br>总成用户手<br>册》图 3-5<br>灵巧手腕部<br>工作空间图 | 详情参考<br>《Synca XFS<br>一体化臂手<br>总成用户手<br>册》图 3-5<br>灵巧手腕部<br>工作空间图 | °  |
| 限位下限 | -178 | -120 | -178 | -60 | -178 | 详情参考<br>《Synca XFS<br>一体化臂手<br>总成用户手<br>册》图 3-5<br>灵巧手腕部<br>工作空间图 | 详情参考<br>《Synca XFS<br>一体化臂手<br>总成用户手<br>册》图 3-5<br>灵巧手腕部<br>工作空间图 | °  |
| 速度上限 | 175  | 175  | 270  | 300 | 240  | 75.8                                                              | 32.5                                                              | %s |

## 6.2 问题排查

### 6.2.1 网络连接问题

实时运动指令和状态信息都通过 UDP 协议单播发送。如果出现“超时前未收到机器人状态反馈”的实时状态异常，请检查本机防火墙的 UDP 进站规则是否为允许状态。

Linux 系统（Debian 发行版）：打开终端，输入命令 `nc -vul -p 1337`，然后再任意运行一个实时模式控制的示例程序，查看是否有来自机器人地址的端口连接过来，并且收到了数据。

### 6.2.2 实时模式直接力矩控制下坠问题

力控和机器本身硬件状态有关系。如果出现开启直接力矩控制之后没有发送任何指令，但是机器人下坠或者发飘等问题，可能是由于力控模型不准的原因。首先需要确认每项力控参数是否准确；其次可以根据实机情况调整发送的运动指令。

以下示例基于对外 SDK `xynova::armhandRobot` (C++, Linux)。  
注：以下为代码片段，只展示核心使用逻辑，仅供实际编程参考。

## 7.1 只读巡检

```
#include <iostream>
#include <vector>
#include "xynova/cpp/armhand/Robot.hpp"
using namespace xynova::armhandRobot;
int main() {
    Robot robot;
    if (robot.Connect() != ErrorCode::Success) return 1;
    SafetyStatus s;
    robot.GetSafetyStatus(s);
    std::cout << "estop=" << s.estop_active
                << " arm_online=" << s.arm_connected
                << " hand_online=" << s.hand_connected << "\n";
    std::vector<JointState> js;
    if (robot.Arm().GetJointStates(js) == ErrorCode::Success)
        for (const auto& j : js)
            std::cout << "J" << int(j.joint_index) << " "
                        << j.position_rad << " rad " << j.temperature_c << " C\n";
    std::vector<FingerState> fs;
    if (robot.Hand().GetFingerStates(fs) == ErrorCode::Success)
        std::cout << "fingers=" << fs.size() << "\n";
    robot.Disconnect();
}
```

## 7.2 机械臂运动（运动前确认安全）

```
robot.Arm().EnableArm(true);
robot.Arm().SetOperatingMode(OperatingMode::ARMHAND_OPERATING_MODE_AUTOMATIC);
robot.Arm().Home();
while (robot.Arm().IsInMotion()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
JointPosition jp{}; // 长度 7 的关节角 (rad)
robot.Arm().MoveAbsJ(jp);
while (robot.Arm().IsInMotion()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
```

## 7.3 灵巧手控制

```
robot.Hand().Enable(true);
robot.Hand().SetControlMode(HandControlMode::ARMHAND_HAND_CONTROL_MODE_POSITION);
std::vector<double> angles(6, 0.2); // rad, 小幅
robot.Hand().SetFingerAngles(angles);
std::vector<FingerState> fingers;
robot.Hand().GetFingerStates(fingers);
```

## 7.4 臂手协同

```
ArmTarget at; at.has_joint_position = true; // 目标关节角
HandTarget ht; ht.has_finger_positions = true;
robot.SynchronizeAction(at, ht); // 阻塞到协同完成
```

## 7.5 更多可运行示例

随包 example/ 目录：armhand\_demo.cpp、test\_misc.cpp、controller\_upgrade.cpp 等可运行 C++ 示例源码。

## CHAPTER 8

---

### 反馈与勘误

---

文档中若出现不准确的描述或者错误，恳请读者指正批评。如果您在阅读过程中发现任何问题或者有想提出的意见，可以发送邮件到 [xuqiu@xynova.com](mailto:xuqiu@xynova.com)，我们的同事会尽量一一回复。

本附录给出门面 SDK `xynova::armhandRobot` 的配置项、枚举类型与数据结构参考；内容取自公开头文件，为权威来源。

## 9.1 配置（RobotConfig）

整机配置类型 `xynova::armhandRobot::RobotConfig`（Types.hpp）；默认构造即出厂默认值，`Robot()` 无参构造等价于全默认配置。

| 段     | 字段                    | 类型          | 默认值           | 说明                              |
|-------|-----------------------|-------------|---------------|---------------------------------|
| arm   | controller_ip         | std::string | 192.168.1.160 | 臂控制器 IP                         |
|       | local_ip              | std::string | ""（自动）        | 本机绑定 IP，实时模式建议显式指定              |
|       | model                 | std::string | XynovaRobot   | 机型标识                            |
|       | auth_key              | std::string | ""            | 鉴权密钥，无鉴权留空                      |
| hand  | arm_side              | ArmSide     | Right         | 本地数值 FK/IK 使用的臂侧别（Left/Right）   |
|       | can_type              | std::string | SockCAN       | CAN 子类型（SockCAN / ChuangXinCAN） |
|       | can_interface         | std::string | can11         | SocketCAN 接口名（ChuangXinCan 时忽略） |
|       | can_id                | uint32_t    | 0xFFFFFFFF    | CAN 从站 ID（出厂默认从站 ID）            |
|       | device_name           | std::string | right_hand    | 设备逻辑名                           |
|       | enable_tactile_wrench | bool        | true          | 指尖六维力周期订阅                       |
|       | enable_tactile_taxel  | bool        | true          | 阵列触觉周期订阅                        |
|       | enable_proximity      | bool        | true          | DTOF 距离周期订阅                     |
| error | enable_detailed_trace | bool        | true          | 记录详细调用栈                         |
|       | max_trace_depth       | uint32_t    | 10            | 调用栈最大深度                         |
|       | log_to_file           | bool        | true          | 错误日志持久化                         |

| 段 | 字段            | 类型          | 默认值          | 说明   |
|---|---------------|-------------|--------------|------|
|   | log_directory | std::string | ./error_logs | 日志目录 |

独立使用 *Arm / Hand* 时不经过 *RobotConfig*：C++ 分别调 *Arm::Connect(controller\_ip, port, auth\_key, connection\_timeout\_sec, local\_ip)* 与 *Hand::Connect(can\_type, can\_interface, can\_id, device\_name, toggles)*。

## 9.2 枚举参考

以下枚举为门面 SDK 对外类型；C++ 中可直接用枚举名（using 自 C 枚举），枚举值与 C ABI 完全一致。

### 9.2.1 ConnectionStatus（C: ArmhandConnectionStatus）

连接状态（整机 / 分端通用）

| 枚举值                                        | 数值 | 含义   |
|--------------------------------------------|----|------|
| ARMHAND_CONNECTION_STATUS_DISCONNECTED     | 0  | 未连接  |
| ARMHAND_CONNECTION_STATUS_CONNECTING       | 1  | 连接中  |
| ARMHAND_CONNECTION_STATUS_CONNECTED        | 2  | 已连接  |
| ARMHAND_CONNECTION_STATUS_HANDSHAKE_FAILED | 3  | 握手失败 |
| ARMHAND_CONNECTION_STATUS_AUTH_FAILED      | 4  | 认证失败 |
| ARMHAND_CONNECTION_STATUS_ERROR            | 5  | 错误   |

### 9.2.2 MotionState（C: ArmhandMotionState）

运动状态机状态（Armhand\_Arm\_getMotionState 返回值）

| 枚举值                              | 数值 | 含义   |
|----------------------------------|----|------|
| ARMHAND_MOTION_STATE_IDLE        | 0  | 空闲   |
| ARMHAND_MOTION_STATE_PLANNING    | 1  | 规划中  |
| ARMHAND_MOTION_STATE_MOVING      | 2  | 运动中  |
| ARMHAND_MOTION_STATE_PAUSED      | 3  | 已暂停  |
| ARMHAND_MOTION_STATE_SERVOING    | 4  | 伺服中  |
| ARMHAND_MOTION_STATE_STOPPING    | 5  | 停止中  |
| ARMHAND_MOTION_STATE_ERROR_STATE | 6  | 错误状态 |

### 9.2.3 RtActiveMode (C: ArmhandRtActiveMode)

实时控制子模式（封装层管理）

| 枚举值                           | 数值 | 含义        |
|-------------------------------|----|-----------|
| ARMHAND_RT_ACTIVE_MODE_NONE   | 0  | 未进入实时控制   |
| ARMHAND_RT_ACTIVE_MODE_SERVOJ | 1  | ServoJ 模式 |
| ARMHAND_RT_ACTIVE_MODE_TORQUE | 2  | 力矩控制模式    |
| ARMHAND_RT_ACTIVE_MODE_FOLLOW | 3  | 点位跟随模式    |

### 9.2.4 RtStateField (C: ArmhandRtStateField)

实时状态数据字段（对应底层 RtSupportedFields）

J6、J7 只支持关节位置获取。

| 枚举值                           | 数值 | 含义                          |
|-------------------------------|----|-----------------------------|
| ARMHAND_RT_FIELD_JOINT_POS    | 0  | 关节位置 (rad)                  |
| ARMHAND_RT_FIELD_JOINT_VEL    | 1  | 关节速度 (rad/s)                |
| ARMHAND_RT_FIELD_JOINT_ACC    | 2  | 关节加速度 (rad/s <sup>2</sup> ) |
| ARMHAND_RT_FIELD_JOINT_TORQUE | 3  | 关节力矩 (N • m)                |

### 9.2.5 OperatingMode (C: ArmhandOperatingMode)

操作模式（manual/automatic）

| 枚举值                              | 数值 | 含义               |
|----------------------------------|----|------------------|
| ARMHAND_OPERATING_MODE_IDLE      | 0  | 未知/未连接           |
| ARMHAND_OPERATING_MODE_MANUAL    | 1  | 手动模式：可拖动，不能跑运动指令 |
| ARMHAND_OPERATING_MODE_AUTOMATIC | 2  | 自动模式：可执行运动指令     |

### 9.2.6 DragSpaceMode (C: ArmhandDragSpaceMode)

拖动空间模式

| 枚举值                                        | 数值 | 含义        |
|--------------------------------------------|----|-----------|
| ARMHAND_DRAG_SPACE_MODE_EXIT               | 0  | 退出拖动      |
| ARMHAND_DRAG_SPACE_MODE_JOINT_SPACE        | 1  | 关节空间拖动    |
| ARMHAND_DRAG_SPACE_MODE_CARTESIAN_X_AXIS   | 2  | X 轴单向拖动   |
| ARMHAND_DRAG_SPACE_MODE_CARTESIAN_Y_AXIS   | 3  | Y 轴单向拖动   |
| ARMHAND_DRAG_SPACE_MODE_CARTESIAN_Z_AXIS   | 4  | Z 轴单向拖动   |
| ARMHAND_DRAG_SPACE_MODE_CARTESIAN_ROTATION | 5  | 笛卡尔旋转方向拖动 |

### 9.2.7 LogCleanupMode (C: ArmhandLogCleanupMode)

日志清理模式

| 枚举值                                 | 数值 | 含义     |
|-------------------------------------|----|--------|
| ARMHAND_LOG_CLEANUP_MODE_COMPLETE   | 0  | 完整目录清理 |
| ARMHAND_LOG_CLEANUP_MODE_TIME_BASED | 1  | 按时间清理  |

### 9.2.8 SyncStrategy (C: ArmhandSyncStrategy)

同步策略

| 枚举值                            | 数值 | 含义            |
|--------------------------------|----|---------------|
| ARMHAND_SYNC_STRATEGY_NONE     | 0  | 无             |
| ARMHAND_SYNC_STRATEGY_SOFTWARE | 1  | 软件同步（主机时间戳对齐） |

### 9.2.9 HandControlMode (C: ArmhandHandControlMode)

灵巧手关节控制模式

| 枚举值                                | 数值 | 含义         |
|------------------------------------|----|------------|
| ARMHAND_HAND_CONTROL_MODE_POSITION | 0  | 位置控制（默认）   |
| ARMHAND_HAND_CONTROL_MODE_FORCE    | 1  | 力控制（固件支持时） |

### 9.2.10 LogLevel (C: ArmhandLogLevel)

控制器日志等级

| 枚举值                       | 数值 | 含义 |
|---------------------------|----|----|
| ARMHAND_LOG_LEVEL_INFO    | 0  | 通知 |
| ARMHAND_LOG_LEVEL_WARNING | 1  | 警告 |
| ARMHAND_LOG_LEVEL_ERROR   | 2  | 错误 |

### 9.2.11 OperationState (C: ArmhandOperationState)

机器人当前运行状态

| 枚举值                                       | 数值   | 含义          |
|-------------------------------------------|------|-------------|
| ARMHAND_OPERATION_STATE_IDLE              | 0    | 机器人静止       |
| ARMHAND_OPERATION_STATE_JOG               | 1    | jog 状态（未运动） |
| ARMHAND_OPERATION_STATE_RT_CONTROLLING    | 2    | 实时模式控制中     |
| ARMHAND_OPERATION_STATE_DRAG              | 3    | 拖动已开启       |
| ARMHAND_OPERATION_STATE_RL_PROGRAM        | 4    | RL 工程运行中    |
| ARMHAND_OPERATION_STATE_DEMO              | 5    | Demo 演示中    |
| ARMHAND_OPERATION_STATE_DYNAMIC_IDENTIFY  | 6    | 动力学辨识中      |
| ARMHAND_OPERATION_STATE_FRICTION_IDENTIFY | 7    | 摩擦力辨识中      |
| ARMHAND_OPERATION_STATE_LOAD_IDENTIFY     | 8    | 负载辨识中       |
| ARMHAND_OPERATION_STATE_MOVING            | 9    | 机器人运动中      |
| ARMHAND_OPERATION_STATE_JOGGING           | 10   | Jog 运动中     |
| ARMHAND_OPERATION_STATE_UNKNOWN           | 0xFF | 未知（发生异常）    |

### 9.2.12 PowerState (C: ArmhandPowerState)

上下电及急停状态

| 枚举值                       | 数值 | 含义    |
|---------------------------|----|-------|
| ARMHAND_POWER_STATE_ON    | 0  | 上电    |
| ARMHAND_POWER_STATE_OFF   | 1  | 下电    |
| ARMHAND_POWER_STATE_ESTOP | 2  | 急停被按下 |

| 枚举值                         | 数值   | 含义         |
|-----------------------------|------|------------|
| ARMHAND_POWER_STATE_GSTOP   | 3    | 安全门打开/安全停止 |
| ARMHAND_POWER_STATE_UNKNOWN | 0xFF | 未知（发生异常）   |

### 9.2.13 MotionControlMode（C: ArmhandMotionControlMode）

SDK 运动控制模式

| 枚举值                                     | 数值 | 含义            |
|-----------------------------------------|----|---------------|
| ARMHAND_MOTION_CONTROL_MODE_IDLE        | 0  | 空闲            |
| ARMHAND_MOTION_CONTROL_MODE_NRT_COMMAND | 1  | 非实时模式执行运动指令   |
| ARMHAND_MOTION_CONTROL_MODE_NRT_RL_TASK | 2  | 非实时模式运行 RL 工程 |
| ARMHAND_MOTION_CONTROL_MODE_RT_COMMAND  | 3  | 实时模式控制        |

### 9.2.14 RecoverOption（C: ArmhandRecoverOption）

急停恢复选项（Armhand\_Arm\_recoverState 参数）

| 枚举值                                  | 数值 | 含义   |
|--------------------------------------|----|------|
| ARMHAND_RECOVER_OPTION_ESTOP_RECOVER | 1  | 急停恢复 |

### 9.2.16 BackupItem（C: ArmhandBackupItem）

控制器备份项（exportControllerBackup 参数）

| 枚举值                                                  | 数值 | 含义            |
|------------------------------------------------------|----|---------------|
| ARMHAND_BACKUP_ITEM_CUSTOM_CONFIG                    | 0  | 自定义配置         |
| ARMHAND_BACKUP_ITEM_ROBOT_CONFIG                     | 1  | 机器人配置         |
| ARMHAND_BACKUP_ITEM_ROBOT_CONFIG_WITHOUT_BODY_PARAMS | 2  | 机器人配置, 不含本体参数 |
| ARMHAND_BACKUP_ITEM_CONTROLLER_LOG                   | 3  | 控制器日志         |
| ARMHAND_BACKUP_ITEM_SYSTEM_LOG                       | 4  | 系统日志          |
| ARMHAND_BACKUP_ITEM_RL_PROGRAM                       | 5  | RL 程序         |
| ARMHAND_BACKUP_ITEM_BACKUP_RL_PROGRAM                | 6  | 备份 RL 工程      |

| 枚举值                                     | 数值 | 含义            |
|-----------------------------------------|----|---------------|
| ARMHAND_BACKUP_ITEM_<br>SDK_RECORD_PATH | 7  | SDK 录制的拖动示教路径 |

### 9.2.17 StopLevel (C: ArmhandStopLevel)

碰撞后机器人停止行为（enableCollisionDetection 参数）

| 枚举值                                | 数值 | 含义                               |
|------------------------------------|----|----------------------------------|
| ARMHAND_STOP_LEVEL_<br>SAFETY_STOP | 0  | stop1: 规划停止后断电，停在原路径上（工业机型仅支持此项） |
| ARMHAND_STOP_LEVEL_<br>PAUSE       | 1  | stop2: 规划停止后不断电，停在原路径上，可恢复       |
| ARMHAND_STOP_LEVEL_<br>SUPPLE_STOP | 2  | 柔顺停止（仅协作机型）                      |

### 9.2.18 FirmwareUpdateState (C: ArmhandFirmwareUpdateState)

固件升级状态机（ArmhandFirmwareUpdateProgress.state）

| 枚举值                                                 | 数值 | 含义     |
|-----------------------------------------------------|----|--------|
| ARMHAND_FIRMWARE_<br>UPDATE_STATE_IDLE              | 0  | 空闲     |
| ARMHAND_FIRMWARE_<br>UPDATE_STATE_CHECKING          | 1  | 检查中    |
| ARMHAND_FIRMWARE_<br>UPDATE_STATE_DOWNLOADING       | 2  | 下载中    |
| ARMHAND_FIRMWARE_<br>UPDATE_STATE_BACKING_<br>UP    | 3  | 备份中    |
| ARMHAND_FIRMWARE_<br>UPDATE_STATE_UPDATING_<br>ARM  | 4  | 升级臂固件中 |
| ARMHAND_FIRMWARE_<br>UPDATE_STATE_UPDATING_<br>HAND | 5  | 升级手固件中 |
| ARMHAND_FIRMWARE_<br>UPDATE_STATE_VERIFYING         | 6  | 校验中    |
| ARMHAND_FIRMWARE_<br>UPDATE_STATE_ROLLING_<br>BACK  | 7  | 回滚中    |
| ARMHAND_FIRMWARE_<br>UPDATE_STATE_COMPLETED         | 8  | 完成     |
| ARMHAND_FIRMWARE_<br>UPDATE_STATE_FAILED            | 9  | 失败     |

## 9.3 数据结构

字段名在 C++ 与 C 中一致；字符串为定长数组，时间戳 `uint64_t`（微秒），`bool` 在 C ABI 为 `int`。

### 9.3.1 Pose（C: ArmhandPose）

笛卡尔位姿：XYZ 位置 + RxRyRz 姿态 + `armAngle` 冗余自由度

| 字段              | 类型                  | 说明          |
|-----------------|---------------------|-------------|
| <code>x</code>  | <code>double</code> | 位置 X (m)    |
| <code>y</code>  | <code>double</code> | 位置 Y (m)    |
| <code>z</code>  | <code>double</code> | 位置 Z (m)    |
| <code>rx</code> | <code>double</code> | 旋转 Rx (rad) |
| <code>ry</code> | <code>double</code> | 旋转 Ry (rad) |
| <code>rz</code> | <code>double</code> | 旋转 Rz (rad) |

### 9.3.2 JointState（C: ArmhandJointState）

单关节状态

| 字段                          | 类型                   | 说明           |
|-----------------------------|----------------------|--------------|
| <code>joint_index</code>    | <code>uint8_t</code> | 关节索引 [0,6]   |
| <code>position_rad</code>   | <code>double</code>  | 关节角度 (rad)   |
| <code>velocity_rad_s</code> | <code>double</code>  | 关节速度 (rad/s) |
| <code>current_a</code>      | <code>double</code>  | 关节电流 (A)     |
| <code>temperature_c</code>  | <code>double</code>  | 关节温度 (°C)    |
| <code>torque_nm</code>      | <code>double</code>  | 关节扭矩 (N · m) |

### 9.3.3 JointPosition（C: ArmhandJointPosition）

关节位置目标（7 轴）

| 字段                                      | 类型                  | 说明          |
|-----------------------------------------|---------------------|-------------|
| <code>joints[ARMHAND_MAX_JOINTS]</code> | <code>double</code> | 各关节角度 (rad) |

### 9.3.4 FingerState（C: ArmhandFingerState）

灵巧手手指状态

| 字段                          | 类型                    | 说明             |
|-----------------------------|-----------------------|----------------|
| <code>finger_index</code>   | <code>uint8_t</code>  | 手指索引           |
| <code>position_rad</code>   | <code>double</code>   | 手指关节角度 (rad)   |
| <code>velocity_rad_s</code> | <code>double</code>   | 手指关节速度 (rad/s) |
| <code>force_n</code>        | <code>double</code>   | 手指输出力 (N)      |
| <code>temperature_c</code>  | <code>double</code>   | 驱动温度 (°C)      |
| <code>error_code</code>     | <code>uint32_t</code> | 手指错误码          |
| <code>is_enabled</code>     | <code>int</code>      | 是否使能           |
| <code>is_in_fault</code>    | <code>int</code>      | 是否故障           |

### 9.3.5 WorldFrame (C: ArmhandWorldFrame)

世界坐标系

| 字段                      | 类型          | 说明           |
|-------------------------|-------------|--------------|
| origin                  | ArmhandPose | 世界系原点相对基座的位姿 |
| name[ARMHAND_STR_SHORT] | char        | 坐标系名称        |

### 9.3.6 CollisionState (C: ArmhandCollisionState)

碰撞状态（仅上报是否发生碰撞及受影响关节）

| 字段                                  | 类型       | 说明      |
|-------------------------------------|----------|---------|
| occurred                            | int      | 是否发生碰撞  |
| affected_joints[ARMHAND_MAX_JOINTS] | uint8_t  | 受影响关节索引 |
| affected_joint_count                | uint32_t | 受影响关节数  |

### 9.3.7 ErrorInfo (C: ArmhandErrorInfo)

错误/故障信息（统一表达，severity>=4 视为故障级）

| 字段                           | 类型          | 说明                  |
|------------------------------|-------------|---------------------|
| error_code                   | uint32_t    | 错误码                 |
| description[ARMHAND_STR_MID] | char        | 人类可读描述              |
| severity                     | uint8_t     | 严重级别 1-10           |
| source                       | ErrorSource | 错误来源                |
| timestamp_us                 | uint64_t    | 发生时间（Unix epoch 微秒） |
| is_active                    | int         | 是否仍活动（未被清除）         |
| has_trace_id                 | int         | trace_id 是否有效       |
| trace_id                     | uint64_t    | 关联的错误追踪 ID          |

### 9.3.8 VersionInfo (C: ArmhandVersionInfo)

版本信息（SDK/臂/手/控制器全版本集）

| 字段                                       | 类型   | 说明          |
|------------------------------------------|------|-------------|
| sdk_version[ARMHAND_STR_SHORT]           | char | 臂手一体 SDK 版本 |
| arm_sdk_version[ARMHAND_STR_SHORT]       | char | 臂底层 SDK 版本  |
| hand_sdk_version[ARMHAND_STR_SHORT]      | char | 手底层 SDK 版本  |
| controller_firmware[ARMHAND_STR_SHORT]   | char | 控制器固件版本     |
| wrist_driver_firmware[ARMHAND_STR_SHORT] | char | 腕驱动固件版本     |

| 字段                                      | 类型   | 说明        |
|-----------------------------------------|------|-----------|
| hand_driver_firmware[ARMHAND_STR_SHORT] | char | 灵巧手驱动固件版本 |
| build_time[ARMHAND_STR_SHORT]           | char | 编译时间      |

### 9.3.9 HandSchedulingToggles (C: ArmhandHandSchedulingToggles)

手侧周期数据订阅开关（决定连接时注册哪些周期命令）

| 字段             | 类型  | 说明                   |
|----------------|-----|----------------------|
| tactile_wrench | int | 触觉 wrench（指尖六维力）周期订阅 |
| tactile_taxel  | int | 触觉 taxel 阵列周期订阅      |
| proximity      | int | 接近觉（DToF 距离）周期订阅     |

### 9.3.10 DevicePartInfo (C: ArmhandDevicePartInfo)

单端设备信息（臂/手通用）

| 字段                                  | 类型       | 说明            |
|-------------------------------------|----------|---------------|
| product_model[ARMHAND_STR_SHORT]    | char     | 产品型号          |
| product_sn[ARMHAND_STR_SHORT]       | char     | 产品序列号         |
| hardware_version[ARMHAND_STR_SHORT] | char     | 硬件版本          |
| software_version[ARMHAND_STR_SHORT] | char     | 软件版本（固件版本）    |
| device_type                         | uint32_t | 设备类型（SDK 枚举值） |
| valid                               | int      | 本次查询是否成功      |

### 9.3.11 DevicePartError (C: ArmhandDevicePartError)

单端错误状态（臂/手通用）

| 字段                         | 类型               | 说明        |
|----------------------------|------------------|-----------|
| timestamp_us               | uint64_t         | 采样时间戳（us） |
| errors[ARMHAND_MAX_ERRORS] | ArmhandErrorInfo | 错误条目列表    |
| error_count                | uint32_t         | 有效条目数     |
| valid                      | int              | 本次查询是否成功  |

### 9.3.12 DeviceInfo (C: ArmhandDeviceInfo)

整机设备信息（臂 + 手聚合）

| 字段   | 类型                    | 说明     |
|------|-----------------------|--------|
| arm  | ArmhandDevicePartInfo | 臂端设备信息 |
| hand | ArmhandDevicePartInfo | 手端设备信息 |

### 9.3.13 ErrorState (C: ArmhandErrorState)

整机错误状态（臂 + 手聚合，timestamp\_us 取两端较新者）

| 字段           | 类型                     | 说明          |
|--------------|------------------------|-------------|
| timestamp_us | uint64_t               | 整机采样时间戳（us） |
| arm          | ArmhandDevicePartError | 臂端错误状态      |
| hand         | ArmhandDevicePartError | 手端错误状态      |

### 9.3.14 DeviceConnectionStatus (C: ArmhandDeviceConnectionStatus)

臂/手分端连接状态（可表达「臂通、手断」等组合）

| 字段   | 类型                      | 说明    |
|------|-------------------------|-------|
| arm  | ArmhandConnectionStatus | 臂链路状态 |
| hand | ArmhandConnectionStatus | 手链路状态 |

### 9.3.15 TactileWrench (C: ArmhandTactileWrench)

单个触觉区域的 Wrench（合力/合力矩，力 N，力矩 N·m）

| 字段           | 类型       | 说明                  |
|--------------|----------|---------------------|
| timestamp_us | uint64_t | 采样时间戳（us）           |
| region       | uint32_t | 触觉区域 ID             |
| dimension    | uint8_t  | Wrench 有效维数（SDK 透传） |
| wrench[6]    | double   |                     |

### 9.3.16 TactileTaxel (C: ArmhandTactileTaxel)

单个触觉点（Taxel）的 Wrench

| 字段           | 类型       | 说明        |
|--------------|----------|-----------|
| timestamp_us | uint64_t | 采样时间戳（us） |
| region       | uint32_t | 触觉区域 ID   |
| taxel_id     | uint16_t | 区域内触觉点编号  |
| dimension    | uint8_t  | 有效维数      |
| wrench[6]    | double   |           |

### 9.3.17 RangePoint (C: ArmhandRangePoint)

单距离点测量

| 字段           | 类型       | 说明                   |
|--------------|----------|----------------------|
| timestamp_us | uint64_t | 采样时间戳 (us)           |
| id           | uint16_t | 点编号                  |
| status       | uint8_t  | 测量状态 (SDK 透传, 0= 有效) |
| distance_m   | float    | 距离 (米)               |

### 9.3.18 ArmTarget (C: ArmhandArmTarget)

臂目标 (关节空间或笛卡尔空间, 至少设置一种)

| 字段                             | 类型                   | 说明             |
|--------------------------------|----------------------|----------------|
| has_joint_position             | int                  | 是否设置关节空间目标     |
| joint_position                 | ArmhandJointPosition | 关节空间目标         |
| has_cartesian_pose             | int                  | 是否设置笛卡尔空间目标    |
| cartesian_pose                 | ArmhandPose          | 笛卡尔空间目标        |
| target_name[ARMHAND_STR_SHORT] | char                 | 目标名称 (用于日志和追踪) |

### 9.3.19 HandTarget (C: ArmhandHandTarget)

手目标 (手指位置或力, 至少设置一种)

| 字段                                    | 类型       | 说明         |
|---------------------------------------|----------|------------|
| has_finger_positions                  | int      | 是否设置手指位置   |
| finger_positions[ARMHAND_MAX_FINGERS] | double   | 手指位置 (rad) |
| finger_position_count                 | uint32_t | 有效手指位置数    |
| has_finger_forces                     | int      | 是否设置手指力    |
| finger_forces[ARMHAND_MAX_FINGERS]    | double   | 手指力 (N)    |
| finger_force_count                    | uint32_t | 有效手指力数     |

### 9.3.20 RobotArmState (C: ArmhandRobotArmState)

整机状态快照-臂端子状态

| 字段                                | 类型                    | 说明     |
|-----------------------------------|-----------------------|--------|
| joints[ARMHAND_MAX_JOINTS]        | ArmhandJointState     | 关节完整状态 |
| joint_count                       | uint32_t              | 有效关节数  |
| is_enabled                        | int                   | 臂是否使能  |
| is_in_motion                      | int                   | 臂是否运动中 |
| collision_state                   | ArmhandCollisionState | 碰撞状态   |
| active_faults[ARMHAND_MAX_ERRORS] | ArmhandErrorInfo      | 活动故障列表 |
| active_fault_count                | uint32_t              | 活动故障数  |

### 9.3.21 RobotHandState (C: ArmhandRobotHandState)

整机状态快照-手端子状态

| 字段                                 | 类型                 | 说明     |
|------------------------------------|--------------------|--------|
| fingers[ARMHAND_MAX_FINGERS]       | ArmhandFingerState | 全指状态   |
| finger_count                       | uint32_t           | 有效手指数  |
| current_gesture[ARMHAND_STR_SHORT] | char               | 当前手势   |
| is_enabled                         | int                | 手是否使能  |
| active_faults[ARMHAND_MAX_ERRORS]  | ArmhandErrorInfo   | 活动故障列表 |
| active_fault_count                 | uint32_t           | 活动故障数  |

### 9.3.22 RobotSyncState (C: ArmhandRobotSyncState)

整机状态快照-同步子状态

| 字段                | 类型                  | 说明               |
|-------------------|---------------------|------------------|
| is_synchronized   | int                 | 是否已同步            |
| timing_error      | double              | 时序误差（秒）          |
| active_strategy   | ArmhandSyncStrategy | 生效的同步策略          |
| last_sync_time_us | uint64_t            | 上次同步时间（epoch 微秒） |

### 9.3.23 RobotState (C: ArmhandRobotState)

整机状态快照（聚合臂/手/同步/版本/错误）

| 字段                                | 类型                      | 说明               |
|-----------------------------------|-------------------------|------------------|
| device_id[ARMHAND_STR_SHORT]      | char                    | 设备 ID            |
| connection_status                 | ArmhandConnectionStatus | 连接状态             |
| operating_mode                    | ArmhandOperatingMode    | 操作模式             |
| arm                               | ArmhandRobotArmState    | 臂状态              |
| hand                              | ArmhandRobotHandState   | 手状态              |
| sync                              | ArmhandRobotSyncState   | 同步状态             |
| version                           | ArmhandVersionInfo      | 版本信息             |
| active_errors[ARMHAND_MAX_ERRORS] | ArmhandErrorInfo        | 活动错误聚合           |
| active_error_count                | uint32_t                | 活动错误数            |
| timestamp_us                      | uint64_t                | 主机侧时间戳（epoch 微秒） |
| controller_timestamp              | uint64_t                | 控制器侧时间戳          |
| arm_timestamp                     | uint64_t                | 臂侧时间戳            |
| hand_timestamp                    | uint64_t                | 手侧时间戳            |

### 9.3.25 SoftLimitConfig (C: ArmhandSoftLimitConfig)

软限位配置 (limits[i] = [下限, 上限], 单位弧度)

| 字段                                    | 类型     | 说明                                |
|---------------------------------------|--------|-----------------------------------|
| enabled                               | int    | 是否使能软限位                           |
| limits[ARMHAND_MAX_JOINTS][2]         | double | 每轴 [下限, 上限]                       |
| reduced_limits[ARMHAND_MAX_JOINTS][2] | double | 每轴缩减限位 [下限, 上限]。各轴 limits 须小于等于此值 |

### 9.3.26 SafetyStatus (C: ArmhandSafetyStatus)

安全系统当前状态

| 字段                  | 类型                       | 说明              |
|---------------------|--------------------------|-----------------|
| arm_connected       | int                      | 臂链路是否在线         |
| hand_connected      | int                      | 手链路是否在线         |
| arm_enabled         | int                      | 臂是否使能 (上电)      |
| hand_enabled        | int                      | 手是否使能           |
| power_state         | ArmhandPowerState        | 上下电/急停状态        |
| operation_state     | ArmhandOperationState    | 当前运行状态          |
| operating_mode      | ArmhandOperatingMode     | 手动/自动模式         |
| motion_control_mode | ArmhandMotionControlMode | 运动控制模式          |
| soft_limit          | ArmhandSoftLimitConfig   | 软限位配置           |
| collision_state     | ArmhandCollisionState    | 碰撞状态            |
| estop_active        | int                      | 是否处于急停          |
| safety_stop_active  | int                      | 是否处于安全停止 (含安全门) |

### 9.3.27 HandConfig (C: ArmhandHandConfig)

手侧配置 (对照 Flex2 灵巧手出厂配置; 字符串 NULL = 取默认值)

| 字段                    | 类型           | 说明                                |
|-----------------------|--------------|-----------------------------------|
| can_type              | const char * | CAN 类型 (如"SockCAN")               |
| can_interface         | const char * | SockCAN 接口名 (如"can11")            |
| can_id                | uint32_t     | CAN 从站 ID (0xFFFFFFFF= 出厂默认从站 ID) |
| device_name           | const char * | 设备逻辑名 (如"right_hand")             |
| enable_tactile_wrench | int          | 触觉 wrench 周期订阅                    |
| enable_tactile_taxel  | int          | 触觉 taxel 阵列周期订阅                   |
| enable_proximity      | int          | 接近觉周期订阅                           |

### 9.3.28 ErrorConfig（C: ArmhandErrorConfig）

错误追踪配置

| 字段                    | 类型       | 说明        |
|-----------------------|----------|-----------|
| enable_detailed_trace | int      | 是否记录详细调用栈 |
| max_trace_depth       | uint32_t | 调用栈最大深度   |
| log_to_file           | int      | 是否持久化错误日志 |

### 9.3.29 RobotConfig（C: ArmhandRobotConfig）

臂手一体化设备配置。

| 字段    | 类型                 | 说明     |
|-------|--------------------|--------|
| arm   | ArmhandArmConfig   | 臂侧配置   |
| hand  | ArmhandHandConfig  | 手侧配置   |
| error | ArmhandErrorConfig | 错误追踪配置 |

### 9.3.30 FirmwareUpdateConfig

固件升级配置

| 字段                 | 类型                      | 说明            |
|--------------------|-------------------------|---------------|
| arm_firmware_path  | std::string             | 臂固件包本地路径      |
| hand_firmware_path | std::string             | 手固件包本地路径      |
| progress_callback  | ArmhandProgressCallback | 进度回调（可为 NULL） |

### 9.3.31 FirmwareUpdateProgress（C: ArmhandFirmwareUpdateProgress）

固件升级进度

| 字段                             | 类型                         | 说明               |
|--------------------------------|----------------------------|------------------|
| state                          | ArmhandFirmwareUpdateState | 升级状态机            |
| progress_percent               | int32_t                    | 进度百分比            |
| current_step[ARMHAND_STR_MID]  | char                       | 当前步骤描述           |
| has_error_message              | int                        | 是否有错误消息          |
| error_message[ARMHAND_STR_MID] | char                       | 错误消息             |
| start_time_us                  | uint64_t                   | 开始时间（epoch 微秒）   |
| estimated_completion_us        | uint64_t                   | 预计完成时间（epoch 微秒） |

### 9.3.32 ArmhandLogEntry (C: ArmhandLogEntry)

控制器日志条目

| 字段                        | 类型              | 说明    |
|---------------------------|-----------------|-------|
| id                        | int32_t         | 日志 ID |
| timestamp[32]             | char            | 日期及时间 |
| content[ARMHAND_STR_LONG] | char            | 日志内容  |
| repair[ARMHAND_STR_LONG]  | char            | 修复办法  |
| level                     | ArmhandLogLevel | 日志等级  |

### 9.3.33 HandErrorState (C: ArmhandHandErrorState)

手端原始错误状态 (warning/fault 位图 + 按关节错误码)

| 字段                                               | 类型       | 说明         |
|--------------------------------------------------|----------|------------|
| timestamp_us                                     | uint64_t | 采样时间戳 (us) |
| warning_flags                                    | uint32_t | 警告位图       |
| fault_flags                                      | uint32_t | 故障位图       |
| joint_error_codes[ARMHAND_MAX_JOINT_ERROR_CODES] | uint32_t | 按关节错误码     |
| joint_error_code_count                           | uint32_t | 有效错误码数     |